PostHarvest AI — System Report

A Retrieval-Grounded Multilingual Advisory System for Post-Harvest Spoilage Management in Uganda

A complete description of what the system does, how it does it, and why it was built this way

Contents

The table below is a live field. In Word, select it and press F9 (or right-click → Update Field) to build it with page numbers.

Update this field to generate the table of contents.

1. Introduction — What This System Is

1.1 The problem it addresses

A farmer in Masindi harvests maize. The crop is good. Six weeks later a third of it is unsellable — mouldy, weevil-damaged, or quietly contaminated with aflatoxin, a toxin produced by fungi that grow on grain stored while still damp. The loss did not happen in the field. It happened in the gap between harvest and market, and it happened for reasons that are, in almost every case, known and preventable.

That knowledge exists. Uganda's National Agricultural Research Organisation publishes on it. The Ministry of Agriculture, Animal Industry and Fisheries issues policy and extension guidance. The Uganda Bureau of Statistics quantifies the losses. The African Postharvest Losses Information System tracks them across the continent. Peer-reviewed research on aflatoxin management in East African maize and groundnuts runs to thousands of pages.

The problem is not that the knowledge is missing. The problem is that it is:

- **Scattered.** Across half a dozen institutional websites, in PDFs buried three clicks deep, in research papers behind academic search interfaces.
- **Unsearchable in practice.** A site's own search box matches words, not meaning. A farmer asking "how do I stop my maize going bad in the store" will not match a document titled *Mycotoxin Contamination Dynamics in Smallholder Storage Systems*.
- **Almost entirely in English.** Uganda has more than forty living languages. The people making the storage decision frequently do not read technical English.
- **Written for the wrong reader.** A forty-page research paper is not an answer to a question asked at seven in the evening with a phone in one hand.

The distance between "the answer exists in a PDF on a government website" and "the person with the problem has the answer" is the gap this system is built to close.

1.2 What was built, in one paragraph

PostHarvest AI is a system that automatically finds, reads, and organises post-harvest guidance from Ugandan and regional agricultural sources; converts that material into a form that can be searched by *meaning* rather than by keyword; and then answers questions about it — in English or five Ugandan languages, over a web browser or WhatsApp — by retrieving the most relevant passages and having an AI language model compose an answer *from those passages only*, with citations back to the original

documents. Around that core it adds a browsable public library of the source documents, a curated catalogue of previously reviewed answers that can be served instantly and free of charge, automated measurement of how faithful each answer is to its sources, and an administrative console through which a human curator controls what enters the knowledge base and what reaches the public.

1.3 Who uses it

The system serves three distinct audiences, and much of its design follows from the fact that their needs differ sharply.

Farmers and extension workers. They reach the system over WhatsApp, often on inexpensive phones with intermittent connectivity and metered data. They ask short, practical questions, frequently in Luganda, Acholi, Runyankole, Ateso or Lugbara. They want a short, usable answer, not a literature review. WhatsApp was chosen precisely because it requires no app installation, no account creation, and no learning — it is already the messaging application in daily use.

Researchers, students and programme staff. They reach the system through a web browser. They ask longer, more specific questions in English, and — critically — they need to know *where an answer came from*. For this audience, an uncited answer is worthless. The web interface therefore surfaces every source document behind an answer, with similarity scores, summaries, and links to the source material.

Administrators and curators. They use a protected console to decide what enters the knowledge base, review the quality of the answers being produced, approve or reject entries for the public catalogue, and issue the access codes that control who may consume the (financially costly) live chat capability.

1.4 The core design commitment: "retrieval-grounded"

The phrase *retrieval-grounded* in the dissertation title names the single most important design decision in the system, and it is worth stating plainly what it means and why it matters.

A modern AI language model — the technology behind ChatGPT and similar systems — can produce fluent, confident, well-structured text about almost any subject. It does this by having absorbed enormous quantities of text during training and learning the statistical patterns of how language fits together. What it does *not* have is a reliable way to

distinguish between something it genuinely absorbed and something it is currently constructing because it sounds plausible. When it constructs, the result is called a **hallucination**: a statement that reads exactly like a fact, is delivered with exactly the same confidence as a fact, and is simply untrue.

In most applications this is an annoyance. In agricultural advisory work it is a hazard. An invented drying-moisture threshold, a fabricated pesticide application rate, a plausible-sounding but wrong claim about how long treated grain remains safe — these are the kinds of errors that damage crops, waste money, or harm health. A system that gives a farmer an invented answer is worse than a system that gives no answer at all, because the farmer has no way to tell the difference.

The retrieval-grounded approach removes the model's licence to invent. Instead of asking the model "what do you know about aflatoxin in stored maize?", the system:

1. Searches its own indexed collection of real documents for passages relevant to the question.
2. Places those specific passages in front of the model.
3. Instructs the model, in the strongest terms the interface allows, to answer **using only those passages**, to cite each passage it draws on by number, and to say explicitly when the passages do not contain enough information to answer.
4. Returns the answer together with the identity of every document that contributed to it.

The consequences of this choice ripple through the entire architecture, and much of this report is an account of them:

- The system needs a **document pipeline**, because it can only answer from what it has actually indexed (Section 5).
- It needs **search by meaning**, because keyword search would fail to retrieve the right passages for a colloquially phrased question (Sections 2.2–2.3, 6.3).
- It needs an **honest refusal path**, because when retrieval finds nothing relevant the correct behaviour is to say so, not to improvise (Section 6.4).
- It needs a way to **measure** whether answers actually stayed within their sources, because an instruction is not a guarantee (Section 9).
- It needs **human curation**, because automated quality signals are evidence, not verdicts (Section 8.5).

Grounding is not a feature bolted onto the system. It is the constraint the system is organised around.

1.5 How to read this report

This report assumes no prior technical knowledge. Section 2 explains, from first principles, every concept the rest of the document depends on — what an AI language model is, what an embedding is, what a vector database does, how retrieval-augmented generation works. Readers already familiar with these ideas can skip it; readers who are not should read it before anything else, because the remaining sections use the vocabulary freely.

Section 3 is a plain inventory of what the system can do, written from the user's point of view with no implementation detail. Sections 4 to 14 explain how each part works. Section 15 justifies every significant technology choice against the alternatives. Sections 16 to 18 cover operation, quality assurance, and an honest assessment of the system's limitations. Appendix A is a glossary; where a term is defined in the body it is also collected there.

Where the report describes a threshold, a limit, or a default, that value is the one actually configured in the running system, not an illustrative example.

2. Concepts You Need First

This section explains the ideas the rest of the report is built on. Each concept is introduced with an everyday comparison, then defined precisely, then connected to the specific job it does in this system. Nothing later in the report requires knowledge that is not established here.

2.1 What a Large Language Model actually is

A **Large Language Model** (LLM) is a computer program that has been trained on an enormous quantity of written text — books, articles, websites, documentation — with a single deceptively simple objective: given some text, predict what comes next.

That objective sounds trivial. Its consequences are not. To predict the next word in "the farmer dried the maize before placing it in the..." with any accuracy, a system must acquire a working representation of grain storage. To continue "translate the following sentence into French:" it must acquire something functionally equivalent to translation ability. Trained at sufficient scale, next-word prediction produces a system that can summarise, rephrase, classify, extract structure from prose, follow written instructions, and hold a coherent conversation.

What LLMs are genuinely excellent at, and what this system uses them for:

- **Rephrasing and summarising.** Turning six dense paragraphs of research prose into three clear sentences a farmer can act on.
- **Following instructions.** "Read this document and return its main topics, the crops mentioned, and the districts referenced, formatted as JSON" is a task an LLM performs reliably.
- **Judgement at scale.** "Is this web page URL likely to concern post-harvest agriculture? Score it 0 to 10." A human does this well but slowly; an LLM does it adequately and instantly, across thousands of URLs.
- **Composing an answer from supplied material.** Given five relevant passages and a question, producing a coherent answer that draws on all five.

What LLMs are bad at, and what this system therefore never asks of them:

- **Being a reliable source of fact.** An LLM's training data is frozen at some point in the past, contains errors, and — most importantly — is not retrievable in any auditable way. The model cannot tell you where it "learned" something, because it did not store it as a fact; it stored it as a statistical tendency.

- **Knowing when it does not know.** This is the crux. A model asked a question outside its competence does not, by default, return an error. It returns fluent text.
- **Arithmetic, precise recall of numbers, and citation.** A model asked to cite a source will happily invent a plausible-looking citation.

The system uses LLMs at exactly four points, and in every case for their strengths rather than their knowledge: scoring URLs for relevance during crawling, extracting structured metadata from document text, composing an answer from retrieved passages, and generating follow-up question suggestions. It never asks an LLM a factual question about agriculture without first supplying the documents that contain the answer.

2.2 Embeddings — turning meaning into numbers

This is the concept that makes the whole system possible, and it is worth taking slowly.

The problem. Suppose your document collection contains a passage that says:

> *Mycotoxin proliferation in stored cereals is substantially mitigated by reducing grain moisture content below 13.5% prior to sealed storage.*

And a farmer asks:

> *How do I stop my maize going mouldy in the store?*

These two texts share almost no words. "Mould" does not appear in the passage. "Maize" does not appear — it says "cereals". "Store" appears as "storage". A conventional keyword search — the kind a website search box performs — matches text against text, and would rank this passage very low or miss it entirely. Yet it is precisely the right passage.

The solution. An **embedding model** is a specialised AI model that takes a piece of text and converts it into a long list of numbers — in this system, a list of 1,536 numbers. That list is called a **vector** or an **embedding**.

The numbers themselves are not individually meaningful; no single number corresponds to "is about maize". What matters is the *arrangement*. The model is trained so that texts with similar meaning produce lists of numbers that are close together, and texts with unrelated meaning produce lists that are far apart — regardless of the specific words used.

An analogy that holds up reasonably well: imagine every possible sentence being assigned a location on an enormous map. The map is not organised geographically but by meaning. All the sentences about grain storage cluster in one region. Within that region, sentences about *moisture and mould* sit in one neighbourhood, sentences about *insect pests* in an

adjacent one. A sentence about football is on a different continent entirely. The embedding — the list of 1,536 numbers — is simply the coordinates of a text on that map. Instead of two coordinates (latitude and longitude) it has 1,536, which is what allows the map to encode such fine distinctions.

Now the earlier problem dissolves. "How do I stop my maize going mouldy in the store?" and "Mycotoxin proliferation in stored cereals is mitigated by reducing grain moisture..." land in nearly the same neighbourhood, because they *mean* nearly the same thing. The system can find one from the other without any word in common.

How similarity is measured. Given two vectors, the system computes their **cosine similarity** — conceptually, how closely the two sets of coordinates point in the same direction. The result is a number, generally between 0 and 1 in this system's usage, where 1 means near-identical meaning and 0 means unrelated. This number is referred to throughout the report as the **similarity score**, and thresholds applied to it govern several important behaviours: whether the system is confident enough to attempt an answer at all (0.30), whether a question matches an existing catalogue entry closely enough to reuse its answer (0.82), and whether two catalogue entries are duplicates of each other (0.90).

Which embedding model. The system's default is OpenAI's `text-embedding-3-small`, which produces 1,536-dimensional vectors. A local alternative, `all-MiniLM-L6-v2`, produces 384-dimensional vectors and runs on the system's own hardware at no cost. Section 15.5 discusses the trade-off. One critical property follows from this: **all vectors in a collection must come from the same model**, because two models place texts on differently organised maps. Changing embedding models means rebuilding the entire index from scratch.

2.3 Vector databases — a filing system for meaning

Having converted texts into vectors, the system needs somewhere to keep them and a fast way to answer the question: *given this query vector, which of my stored vectors are closest to it?*

A conventional database is the wrong tool. PostgreSQL, the system's main database, is superb at questions like "give me every document from NARO discovered in the last month whose status is 'processed'". It is designed around exact matching, sorting, and relationships between records. It has no efficient concept of "nearest in a 1,536-dimensional space".

A **vector database** is a database built specifically for that one question. It stores vectors alongside associated data (in this system: the original text of the passage, the identifier of the document it came from, its source URL, and its classification metadata), and it can find the closest matches to a query vector in milliseconds even across millions of stored vectors. It achieves this using approximate nearest-neighbour indexes — data structures that avoid comparing the query against every stored vector by organising the space so that only a small, promising fraction needs to be examined.

This system supports two vector databases behind a single common interface:

- **Qdrant**, a purpose-built vector database that runs as its own service. This is the production choice.
- **ChromaDB**, which runs *embedded* — as a library inside the application, storing its data in a local directory with no separate service to install or start. This is the development choice, and it means a developer can run the entire system on a laptop without setting up additional infrastructure.

The rest of the system does not know or care which is in use; Section 15.3 explains how that indifference is achieved and why it matters.

2.4 Chunking — why documents are broken into pieces

A vector represents the meaning of a piece of text. But meaning does not compress indefinitely. If you embed an entire sixty-page report on post-harvest losses as a single vector, you get a set of coordinates for "a long document broadly about post-harvest losses in East Africa" — which is true, useless, and equally close to every other long document broadly about post-harvest losses. All the specific, actionable content is averaged away.

The solution is **chunking**: splitting each document into passages small enough that each has a single, coherent topic, and embedding each passage separately. Now the sixty-page report contributes perhaps fifty vectors, one of which represents specifically "the paragraph about acceptable moisture content thresholds for sealed storage" and can be retrieved on its own merits.

Two decisions govern chunking, and this system's choices are:

Chunk size: approximately 1,000 tokens. A *token* is roughly a word-fragment; the system estimates four characters per token, so a 1,000-token chunk is around 4,000 characters — one to two pages of prose. Small enough to be topically focused, large enough to retain the context that makes a passage interpretable.

Overlap: approximately 200 tokens. Consecutive chunks share their boundary region. Without overlap, a passage that begins at the end of chunk 3 and concludes at the start of chunk 4 is fragmented across two vectors, and neither fragment fully represents it — so neither is retrieved when it should be. With 200 tokens of overlap, the complete passage exists intact in at least one chunk. The cost is roughly 20% redundant storage, which is trivial; the benefit is that no idea is lost at a boundary.

The system does not split blindly at character counts. It splits on paragraph boundaries wherever possible, falling back to sentence boundaries when a single paragraph is itself too large. A chunk that begins mid-sentence is a chunk whose meaning has been damaged.

2.5 Retrieval-Augmented Generation — the central mechanism

Retrieval-Augmented Generation (RAG) is the technique that ties Sections 2.1 to 2.4 together, and it is the core of this system. The name describes it accurately: *generation* (an LLM writing an answer) that is *augmented* by *retrieval* (real documents fetched first).

The full loop, for a single question:

5. **The user asks a question.** "How should I store maize to prevent aflatoxin?"
6. **The question is embedded.** The same embedding model that processed the documents converts the question into a 1,536-number vector — coordinates on the same meaning-map.
7. **The vector database is searched.** It returns the passages whose vectors sit closest to the question's vector — in this system, the top 5 — each with a similarity score.
8. **A confidence check is applied.** If even the best match scores below 0.30, the system concludes that nothing in its collection genuinely addresses the question, and stops here (Section 6.4).
9. **A prompt is assembled.** The retrieved passages are formatted, numbered `[1]` through `[5]`, and combined with the question and a system instruction that reads, in part: *answer using only the provided document excerpts; always cite your sources by source number; if the excerpts do not contain enough information, say so clearly — do not fabricate information.*
10. **The LLM composes an answer** from that material and nothing else.
11. **Sources are attached.** The passages that fed the answer are grouped by their source document and returned alongside it, so the user can see and check exactly what the answer was built from.

The properties this buys are the properties the system exists to have. The answer is **current**, because it reflects whatever documents were indexed this morning rather than

whatever the model absorbed during training. It is **verifiable**, because every claim traces to a passage the user can read. It is **bounded**, because the model was given specific material and told to stay inside it. And it is **honest about its limits**, because when retrieval fails the system says so rather than improvising.

RAG is not a perfect defence — an LLM can still misread a passage or over-generalise from it — which is precisely why the system independently measures the faithfulness of every answer against its sources (Section 9).

2.6 Machine translation and language identification

Two related but distinct capabilities are required for multilingual operation.

Language identification answers: *what language is this text in?* Given a message, a language-identification model returns a language code — `lug` for Luganda, `ach` for Acholi, `eng` for English — often with a confidence value. It is a classification task, and it becomes harder with short inputs, mixed-language messages, and closely related languages.

Machine translation answers: *what does this text say in another language?* A translation model converts text from a source language into a target language.

For widely spoken languages, translation is a solved-enough problem. For Uganda's languages it is not: they are, in the terminology of the field, **low-resource** — there is comparatively little parallel text (the same content in two languages) available to train on, so general-purpose translation systems perform poorly or do not support them at all. This system uses **Sunbird AI**, a Ugandan organisation that builds language technology specifically for Ugandan languages, and which supports the five the system offers.

The system uses a **pivot-through-English** strategy, which is worth explaining because it shapes the whole request flow. The document collection is in English. The embedding model was trained predominantly on English. Both facts mean that meaning-based search works best in English. So a question in Luganda is translated to English, searched in English, answered in English, and the English answer is then translated back into Luganda. English acts as the hub through which every language passes. Section 7 covers the consequences of this — including the quality ceiling it imposes, since every translation step introduces some loss.

2.7 Workflow orchestration — scheduling that copes with failure

The document pipeline runs on a schedule: a full crawl every night, incremental checks hourly, retries of failed documents weekly. The naive tool for this is `cron`, the standard Unix scheduler, which runs a command at a set time and does nothing else.

The pipeline needs more than that, because it interacts with the open internet, which is unreliable in specific and predictable ways. A government website is down for maintenance. A PDF link 404s. A server is slow and a request times out. An external API rate-limits. None of these should stop a crawl of five sites; all of them need to be recorded, retried where sensible, and visible afterwards.

Prefect, the orchestration tool used here, provides that. A pipeline is expressed as *flows* (overall processes) composed of *tasks* (individual steps). Prefect then supplies, without any additional code: automatic retries with configurable delays on any task; a persistent record of every run, its inputs, its outputs, and its failures; a web interface for inspecting all of it; and the ability to trigger a run on demand as well as on schedule. Section 5.10 describes how the system uses each of these.

2.8 Supporting vocabulary

These terms appear throughout the report. Each is defined once here.

API (Application Programming Interface). A defined set of requests one program can make of another, and the responses it will get back. This system's backend exposes an HTTP API; the web application and WhatsApp integration are both clients of it.

Endpoint. A single addressable operation within an API, identified by a method and a path — for example `POST /chat` submits a question, `GET /library/documents` lists published documents.

Webhook. An inversion of the usual arrangement: instead of the system repeatedly asking an external service "is there anything new?", the external service calls the system when something happens. WhatsApp messages arrive this way — Meta's servers make an HTTP request to the system's `/webhook/whatsapp` address the moment a user sends a message.

JWT (JSON Web Token). A compact, cryptographically signed token issued after a successful login and presented with each subsequent request as proof of identity. Because the signature can be verified without a database lookup, and because the token carries its

own expiry time, it provides stateless authentication. Administrators receive one valid for eight hours.

SSE (Server-Sent Events) / streaming. A mechanism by which the server holds a connection open and pushes pieces of a response as they become available, instead of waiting until the whole response is ready. This is why answers appear word by word rather than after a ten-second blank pause.

Object storage. A storage system for whole files, addressed by name, with no folder hierarchy or partial-file editing — designed for durability and scale rather than for the read-write access patterns of a filesystem. The system uses MinIO to keep the original bytes of every downloaded document.

ORM (Object-Relational Mapper). A library that lets application code work with database records as ordinary objects instead of hand-written SQL. This system uses SQLAlchemy. It also uses raw SQL deliberately in a handful of places where the database can do something the ORM expresses poorly.

Migration. A versioned, ordered change to the database's structure — adding a column, creating an index — recorded as code so that every deployment of the system can be brought to the same schema state reproducibly. The system uses Alembic; Section 13.8 explains why this matters.

Idempotency. The property that performing an operation twice has the same effect as performing it once. It is what makes retrying safe. Several parts of this system are carefully built to be idempotent, and Section 14.4 explains where and how.

3. What the System Can Do — Capability Catalogue

This section inventories the system's capabilities from the user's point of view, with no implementation detail. It answers the question "what can it do?" directly; the sections that follow answer "how?".

3.1 Browse the document library without any account

Anyone with a web browser can open the system and browse its collection of published source documents. No account, no code, no login.

Each document is presented as a card carrying a title, an AI-generated two-to-three-sentence summary, its category (research paper, extension guide, policy document, statistical report, and so on), and tags for the topics, crops and regions it covers. Opening a document shows the full summary, the list of specific practices extracted from it, indicators for whether it contains actionable advice and whether it contains quantitative data, up to three excerpts of its actual text, and a link to the original source.

The collection can be filtered by **topic** (storage, drying, pest management, contamination, quality control, handling, marketing and others), by **crop** (maize, beans, groundnuts, cassava, coffee and others), and by **region** (Ugandan districts and regions mentioned in the document). Each filter value is shown with a count of matching documents, so the user can see the shape of the collection before filtering it. A text search over titles and summaries is also available.

Every one of these attributes — summary, category, topics, crops, regions, key practices — was generated automatically when the document was ingested. Nobody typed them. Section 5.5 explains how.

Crucially, browsing the library involves **no AI model at all**. It reads metadata that already exists in the database. It is therefore instantaneous and costs nothing to serve, which is why it, rather than the chatbot, is the front door of the system.

3.2 Ask a question and get a reviewed answer instantly

The system's homepage is a single question box. When a user types a question and submits it, the system does not immediately start a chat. It first checks whether it already has a reviewed answer to that question.

This check is semantic, not literal. The question is embedded and compared against a catalogue of previously approved question-and-answer pairs. If a catalogue entry's question is close enough in meaning — a similarity of 0.82 or above — the system returns that entry's answer directly.

What the user gets is superior to a live answer in almost every respect. It is **instant**, because no document search or answer generation is needed. It is **already translated** into the user's language, because approved entries are translated once at approval time into all five Ugandan languages. It carries the **same citations** as the original answer. And it has been **reviewed by a human being**, who saw its automated quality scores and judged it worth publishing.

It also costs nothing to serve and requires no access code.

If the match is good, the user can mark it "this answered it", ask a follow-up (which carries them into live chat with the catalogue answer as context), or say "this doesn't answer my question" — which records the mismatch as evidence and hands them to live chat. If there is no good match, the user is passed to live chat automatically, with their question carried across so they do not retype it.

3.3 Ask a question live and get a cited, grounded answer

When the catalogue has no answer, the full retrieval-augmented pipeline runs. The user sees:

Progress, as it happens. The interface reports each stage in turn — preparing, detecting language, searching, reading sources, synthesising, generating follow-ups — so that a ten-second wait reads as work in progress rather than a frozen screen.

The answer, streaming word by word, formatted with headings, lists and emphasis where the model produced them.

Sources, grouped by document. Rather than showing five disconnected passage fragments, the system groups the retrieved passages by the document they came from and presents one entry per document, carrying its title, the site it came from, its summary, its topics, its highest similarity score, a text snippet, the citation numbers `[1]`, `[3]` that refer to it in the answer, and — where the document is published in the library — a link straight to it.

Suggested follow-up questions. Two to four next questions, blending semantically related entries from the reviewed catalogue with questions generated from the answer and its evidence. These are translated into the user's language.

An honest refusal when appropriate. If nothing relevant was found, the system says so and points the user to the library and catalogue, rather than producing an answer from the model's own recollection.

3.4 Do all of the above in six languages

The system operates in English, **Luganda**, **Acholi**, **Runyankole**, **Ateso** and **Lugbara**.

A user simply writes in their language. The system detects it automatically, translates the question into English for searching, and translates the answer back. The interface shows which language it detected and is responding in, and a picker in the header allows the user to override the detection — useful when a very short message is ambiguous, or when a user wants an answer in a language other than the one they typed in.

Language support extends across the whole product, not just the chat: catalogue answers are stored in all six languages, and follow-up suggestions are translated before display.

3.5 Use it over WhatsApp with no app install

The system is reachable as a WhatsApp contact. A user sends their three-word access code as an ordinary message; the system confirms which group they have been connected to; from then on, every message they send is treated as a question and answered.

This channel is deliberately spare. There is no interface to learn, no application to install, no account to create, and no data cost beyond the messages themselves. Conversations persist, so context carries between messages. The system handles the practical realities of the channel: it recognises and ignores duplicate deliveries when Meta retries a message, it limits each phone number to 30 messages an hour, it replies helpfully to images and voice notes explaining that it can only read text, and it responds within the 15-second budget imposed by Meta's requirement for a fast acknowledgement.

3.6 Continue a conversation

Chat is conversational, not one-shot. The system carries the recent history of a conversation into each new question, so a user can ask "how long can I store it that way?" and be understood.

On the web, past conversations are saved as **threads**. A history panel lists them by their opening question, with message counts and timestamps, and any of them can be reopened and continued. On WhatsApp, the phone number itself identifies a persistent conversation.

3.7 Choose which AI model answers

Web users can choose which language model composes their answers, from those the deployment has configured — typically Groq's Llama 3.3 70B and OpenAI's GPT-4o-mini. The choice is remembered across sessions.

Exposing this was a deliberate decision rather than an oversight. The two models have genuinely different characteristics in speed, cost, phrasing and handling of long conversations, and for a system whose purpose includes evaluating retrieval-grounded advisory quality, letting a user — or a researcher — switch models on the same question and compare is a feature, not a leak of implementation detail.

3.8 Administer the knowledge base

A password-protected console at `/admin` provides:

- **Overview** — corpus size, share of documents successfully processed, number of indexed passages, and crawl activity over the last 30 days.
- **Documents** — a filterable list of everything discovered, with each document's processing outcome explained in plain language; the ability to publish or unpublish a document to the public library, rename it, or delete it entirely (removing its passages from search and its file from storage).
- **Add Document** — ingesting a document directly by drag-and-drop file upload or by URL, bypassing the crawler but running the identical extraction, classification and indexing pipeline.
- **Search** — raw semantic search over the index, showing which passages a query retrieves and with what scores. This is the primary tool for diagnosing why an answer was or was not found.

- **Q&A** — an interactive console running the full retrieval-and-answer pipeline against the index, for testing answer quality without going through the public chat.
- **Chat Quality** — automated faithfulness scores for every answer the system has produced, with distribution statistics and the ability to read the worst-scoring answers alongside their questions.
- **Catalogue** — the review queue for candidate question-answer pairs, with their quality scores, and the controls to approve, reject, retract or retry publication.
- **Access Codes** — issuing codes with monthly message quotas, monitoring usage, adjusting limits, and revoking or reactivating access.
- **Configuration and health** — a read-only view of the running configuration with secrets masked, and live connectivity checks against every dependency.
- **Crawl triggers** — starting a full or incremental crawl on demand.

3.9 What it deliberately does not do

Stating the boundaries is as important as stating the capabilities.

It does not give personalised agronomic prescriptions. It reports what the indexed literature says. It does not know the user's farm, soil, variety, or local conditions, and does not pretend to.

It does not answer outside its corpus. Questions about market prices, weather, credit, or subjects no indexed document covers produce an explicit "I could not find relevant information" rather than a guess.

It does not offer unrestricted public chat. Browsing is open to all; live chat requires an access code and consumes a metered quota. Section 10 explains why this is a design position rather than a limitation.

It does not index everything it finds. Material scoring below 6.0 out of 10 for post-harvest relevance is recorded but never indexed, and a document being indexed does not by itself make it publicly visible — a human must publish it.

It does not verify that its sources are correct. It faithfully reports what its documents say. If a source document is wrong, the system will confidently repeat it. Curation of the corpus is the human's responsibility, and Section 18.2 returns to this.

4. System Architecture — How the Pieces Fit

4.1 The two halves

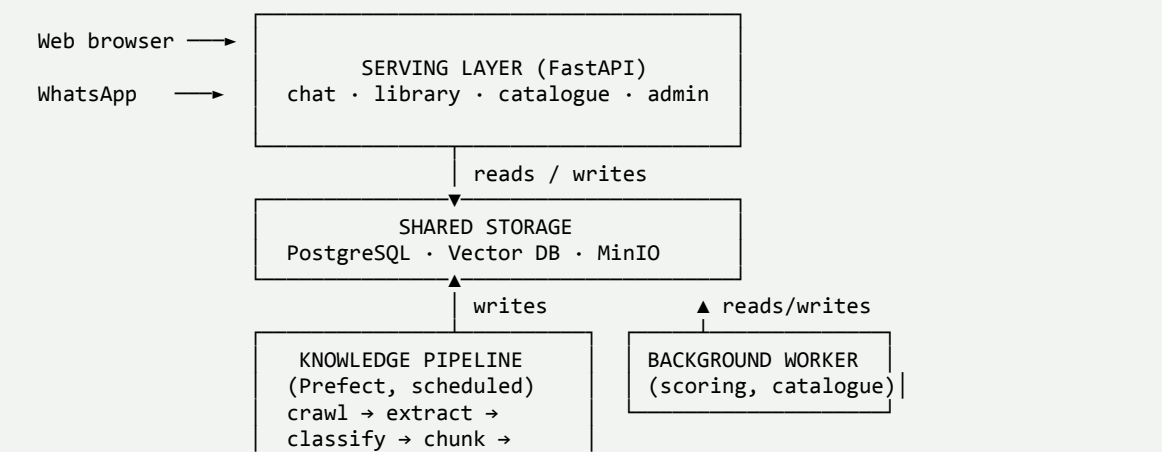
The system is best understood as two largely independent halves that meet only at shared storage.

The knowledge pipeline runs in the background on a schedule. It crawls agricultural websites, downloads documents, extracts their text, classifies them with an LLM, splits them into passages, converts each passage into a vector, and writes the results to storage. No user ever interacts with it directly. It is slow, batch-oriented, tolerant of partial failure, and driven by Prefect.

The serving layer runs continuously and responds to users. It answers questions over web and WhatsApp, serves the public library and catalogue, and hosts the administrative console. It is latency-sensitive, request-driven, and built on FastAPI.

The two never call each other. The pipeline writes to PostgreSQL, MinIO and the vector database; the serving layer reads from them. This separation has a concrete operational payoff: a crawl that fails at two in the morning because a government website is down has no effect whatsoever on a farmer asking a question at seven. Conversely, a spike in chat traffic cannot delay document processing. Each half can be restarted, redeployed or scaled without touching the other.

A third, smaller component sits between them: a **background worker** that handles the slow follow-up work generated by chat — scoring answers for faithfulness, evaluating whether an answer is worth cataloguing, and translating approved catalogue entries into five languages. Section 14 covers it in detail.



```
                    │ embed → index                │
                    └──────────────────────────────┘
```

4.2 The four storage systems and why each exists

A common first reaction to this architecture is that four storage systems is three too many. Each earns its place by doing something the others cannot.

PostgreSQL — facts and relationships. The relational database holds everything that has structure: which documents exist and what state each is in, the classification metadata for each, every chat session and message, access codes and their usage counters, catalogue entries, background jobs, and operational metrics. It is the system's source of truth. It is used because the data genuinely is relational (a document has one classification and many chunks; a session has many messages; an access code has many grants) and because several parts of the design depend on capabilities a simpler store would not provide — transactional guarantees when reserving quota, row-level locking, array columns with efficient indexes, and JSON columns for semi-structured data. Section 15.2 expands on this.

The vector database — search by meaning. Qdrant or ChromaDB holds one vector per passage, with the passage text and its metadata attached. Its single job is answering "which passages are closest in meaning to this query?" quickly. PostgreSQL cannot do this efficiently, and a vector database cannot do what PostgreSQL does, so both exist.

MinIO — the original files. Every downloaded PDF, HTML page or document is stored byte-for-byte in object storage. Nothing in the serving path reads it. It exists for provenance and reprocessing: if the extraction logic improves, or a chunking parameter changes, or a classification prompt is refined, the entire corpus can be reprocessed from the original files without re-crawling the internet — which would take hours, hammer other people's servers, and might not even be possible if a source has since been taken down.

The browser's local storage — session identity. A small amount of state lives in the user's own browser: their session identifier, their access token, their model preference, and a question in transit between the ask gate and live chat. This is not a database, but it is where a specific and necessary piece of state lives, and it is why closing and reopening the browser does not lose the user's access or their conversation.

4.3 End-to-end walkthrough of a single question

Following one WhatsApp question through the whole system makes the architecture concrete. A user sends, in Luganda, a question about protecting stored maize from weevils.

12. **Meta's servers deliver the message** to the system's webhook endpoint over HTTPS.
13. **The request's signature is verified.** The system recomputes a cryptographic signature over the raw request body using a shared secret and compares it with the one Meta supplied. A mismatch means the request did not come from Meta, and it is rejected.
14. **The message is deduplicated.** Meta retries deliveries it believes failed. The system keeps a short-lived record of recently seen message identifiers and silently ignores repeats.
15. **The phone number's rate limit is checked** — no more than 30 messages an hour.
16. **The sender's access grant is resolved.** The phone number is hashed and looked up. If no grant exists, the system replies asking for a three-word access code and stops. If the message *is* a three-word code, it is treated as a redemption attempt instead.
17. **Quota is reserved.** In a single atomic database transaction, the system verifies the group's code is active, that its monthly message allowance is not exhausted, and that the system-wide ceiling has not been reached — then increments both counters. Only after this succeeds does anything expensive begin.
18. **A conversation session is found or created** for that phone number and access grant.
19. **The retrieval pipeline runs**, with a 15-second timeout (Section 6.1 details its steps): language detection identifies Luganda, the question is translated to English, embedded, and used to search the vector database; the top five passages are retrieved; a confidence check passes; the passages, the question and the recent conversation history are sent to the LLM with instructions to answer only from the passages; the English answer is translated back into Luganda.
20. **Both messages are stored** — the user's question and the assistant's answer, along with the detected language, the source documents, and the suggested follow-ups. In the same database transaction, a background job is queued to score the answer's faithfulness later.
21. **The reply is sent** back through Meta's API to the user's phone.
22. **Later, and entirely off the user's critical path**, the background worker picks up the queued job, computes a faithfulness score against the source passages, stores it on the message, and — if the answer is well grounded — asks an LLM whether the exchange

is worth adding to the reviewed catalogue, checks it against existing entries for duplication, and if it survives, files it for a human curator to review.

4.4 End-to-end walkthrough of a single document

The same treatment for a document, from web page to searchable knowledge.

23. **A scheduled crawl begins** at 02:00 UTC. For each enabled site, the crawler starts at the site's home page and follows links, respecting the site's configured depth limit, URL patterns, exclusion patterns, and a politeness delay of about two seconds between requests. It identifies itself honestly by user agent.
24. **Discovered URLs are triaged by an LLM** in batches of twenty. Each is scored 0–10 for likely post-harvest relevance. Those scoring below 6.0 are discarded without ever being downloaded.
25. **Surviving URLs are recorded** in PostgreSQL as documents with status `discovered`.
26. **The processing pipeline picks them up.** Each is downloaded and stored verbatim in MinIO; a SHA-256 hash of its content is computed so the same file arriving by another route is recognised as a duplicate.
27. **Text is extracted** according to file type: PyMuPDF and pdfplumber for PDFs, Trafilatura for web pages, python-docx for Word files, yt-dlp for video metadata and transcripts. If less than 100 characters of usable text emerges, the document is marked `skipped` with an explanation.
28. **The text is classified by an LLM**, which returns topics, crops, regions, target audience, document category, a summary, a list of key practices, flags for actionability and quantitative content, and a final relevance score. This metadata is stored and becomes the backbone of the public library.
29. **The text is chunked** into ~1,000-token passages with ~200-token overlap, split on paragraph and then sentence boundaries.
30. **Each chunk is embedded**, in batches, into a 1,536-number vector.
31. **The results are stored twice** — the chunk text and metadata into PostgreSQL, the vector with its payload into the vector database, linked by a shared identifier.
32. **The document is marked `processed`.** It is now retrievable by the chatbot. It is *not* yet visible in the public library: that requires an administrator to publish it explicitly.

4.5 Process and deployment topology

In a full deployment the following run as separate processes:

Process	Responsibility	Why separate

API (FastAPI/uvicorn)	All HTTP traffic: chat, library, catalogue, admin, WhatsApp webhook	Must stay responsive; nothing slow may run inside it
Frontend (nginx serving built assets)	Delivers the web application to browsers	Static files; no application logic
Catalogue worker	Faithfulness scoring, catalogue candidate evaluation, translation, publication	Jobs take tens of seconds and load a machine-learning model into memory; running them inside the API would block requests and bloat its memory footprint
Prefect server	Schedules, records and displays pipeline runs	Orchestration state must survive pipeline restarts
Prefect worker	Executes crawl and processing flows	Long-running, resource-hungry, and independently restartable
PostgreSQL	Relational storage	—
Qdrant	Vector storage and search	—
MinIO	Object storage	—

The API deliberately ships without the Prefect orchestration stack installed. Its container is therefore substantially smaller and faster to start. The processing code accommodates this: when Prefect is unavailable, its decorators degrade to no-ops so the same functions can be called directly — which is exactly what happens when an administrator uploads a document through the console and expects it indexed immediately rather than at the next scheduled run.

5. The Knowledge Pipeline — How Documents Get In

Everything the system can say is bounded by what this pipeline has collected. It is, in a real sense, the system's most important component: the chatbot is only as good as the corpus behind it.

5.1 Site configuration

Crawling five different websites well is not a matter of pointing a generic crawler at five addresses. Each site has its own structure, its own conventions, and its own pathologies. The system therefore describes each source declaratively, as a configuration object rather than as code.

A site configuration specifies:

Setting	Purpose
url	Where crawling begins
strategy	Crawl approach — recursive link following, search-focused, sitemap-driven, or RSS
max_depth	How many link-hops from the start page to travel
url_patterns	Regular expressions a URL must match to be followed (e.g. anything containing "publication", "research", or ending in .pdf)
exclude_patterns	Regular expressions that disqualify a URL (login pages, admin paths, comment reply links, image galleries)
keywords	Terms indicating relevance for this source
document_patterns	Filename patterns identifying high-value documents
rate_limit	Seconds to wait between requests
requires_js	Whether the site's content is rendered by JavaScript
llm_filter	Whether discovered URLs should be scored by the LLM before storage
headers	Any custom HTTP headers the site requires

The five configured sources:

NARO (National Agricultural Research Organisation, `naro.go.ug`) — recursive crawl to depth 4, targeting publications, research, reports and PDF downloads, excluding administrative and comment URLs. The richest source of Ugandan post-harvest research.

Ministry of Agriculture (`agriculture.go.ug`) — recursive crawl to depth 3, targeting policy, strategy, plans, reports and publications. The source of policy context and national programme documentation.

Agricultural Extension Portal (`extension.agriculture.go.ug`) — recursive crawl to depth 4, targeting advisory content, guides, manuals, training material and best practices. Flagged as requiring JavaScript, which Section 18.2 returns to as a known limitation.

Uganda Bureau of Statistics (`ubos.org`) — a *search-focused* strategy to depth 2 rather than a broad crawl, because the site is large and mostly irrelevant to this domain. It targets specific document patterns: annual agricultural surveys, crop statistics, agricultural production reports, agricultural census documents. This is where quantitative loss figures come from.

APHLIS (African Postharvest Losses Information System, `aphlis.net`) — recursive crawl to depth 3 over news, pages, publications, methodology and training material. Its interactive data tables and maps are explicitly excluded, because they are rendered by JavaScript from an API and contain no crawlable text. Disabled by default and enabled per deployment.

Adding a sixth source requires a configuration entry, a feature flag, and one line in the enabled-sites list. No pipeline code changes.

A per-site SSL exception mechanism also exists: some government sites have expired or misconfigured certificates, and rather than disabling certificate verification globally — which would be a genuine security regression — the system allows specific named sites to be exempted while every other request remains verified.

5.2 Discovery — finding candidate documents

Discovery begins at a site's home page and follows links breadth-first. At each step the crawler fetches a page, parses its HTML, extracts every link, converts relative links to absolute ones, and tests each against the site's rules: it must be on the same domain, must not match any exclusion pattern, and — if inclusion patterns are configured — must match at least one.

Three properties of this crawler are worth noting because they reflect deliberate choices about being a good citizen of other people's infrastructure:

It identifies itself honestly. Every request carries the user agent `PostHarvestBot/1.0 (+https://fourlanes.tech/bot)`, including a URL where an administrator of a crawled site can find out what it is and who runs it. It does not pretend to be a browser.

It waits between requests. Roughly two seconds per request per site, configurable per site. A crawl of a large site therefore takes a long time. That is the correct trade-off: the alternative is placing measurable load on a government agency's web server, and there is no deadline that justifies it.

It bounds itself. Depth limits, URL caps and pattern filters mean the crawler cannot wander indefinitely into a site's archives.

A crawl records itself in the database as a **crawl run** — site, start and end time, status, counts of URLs discovered and documents stored, and any errors — so that every document can later be traced to the crawl that found it.

5.3 LLM URL triage

A crawl of NARO's site may surface several thousand URLs. Most are not post-harvest documents: they are news items, staff pages, event announcements, contact forms. Downloading, extracting and classifying all of them would be slow and expensive.

The system therefore performs an inexpensive triage step *before* downloading anything. Discovered URLs are sent to the LLM in batches of twenty with a prompt asking, for each, a relevance score from 0 to 10, the likely topics, the likely document type, and a priority. The prompt names the indicators to look for — post-harvest, storage, drying, pest, loss, handling, preservation, quality, contamination, aflatoxin, PDF and document files, research publications, extension guides.

URLs scoring below the configured relevance threshold of **6.0** are discarded. Nothing is downloaded, nothing is extracted, nothing is embedded.

Judging a document by its URL alone is obviously imperfect — a URL is a weak signal, and some good documents have uninformative addresses. The design accepts that imprecision knowingly, because the alternative is to spend a full download-extract-classify cycle on every candidate. A URL scored optimistically survives to be judged properly on its content later (Section 5.5), where a second, much better-informed relevance score is applied. The triage step is a cheap filter, not a final verdict, and it is calibrated to be generous rather than strict.

Batching in twenties keeps each response well within token limits and each failure small. If the LLM returns malformed output for a batch, the system does not lose the batch: it assigns every URL in it a neutral score of 5.0 with an explanatory reason, so the failure is visible and recoverable rather than silent.

5.4 Extraction — getting text out of anything

A discovered document might be a PDF, a web page, a Word file, or a video. Each needs different handling, and the system detects type from the URL and the HTTP content type before dispatching to the right extractor.

PDFs — the majority of the valuable material — are handled by two libraries in combination. **PyMuPDF** extracts the text, page by page, in a mode that preserves reading order rather than returning text in arbitrary layout order. **pdfplumber** extracts tables, which matter disproportionately in this domain: loss percentages by region, moisture thresholds by crop, and treatment application rates all live in tables, and a table flattened into prose loses the relationships between its cells.

Web pages are handled by **Trafilatura**, a library built specifically for the problem of separating an article's actual content from the navigation menus, sidebars, cookie banners, footers, related-article widgets and advertisements around it. Naive HTML-to-text conversion produces a document that is 70% navigation, which pollutes the index with meaningless passages that then compete for retrieval. Trafilatura is configured to include tables and to favour recall — preferring to keep borderline content rather than discard it. If it returns nothing usable, a BeautifulSoup-based fallback strips scripts and styling and takes the remaining text.

Word documents are read with python-docx, which walks the paragraphs and additionally flattens each table row into a pipe-delimited line so tabular content survives.

Videos are handled with **yt-dlp**, which retrieves metadata (title, description, uploader, duration, tags) and, where available, the transcript or subtitles. Agricultural extension content increasingly appears as video, and a video with a transcript is a perfectly good text document as far as the pipeline is concerned.

After extraction, the text is **cleaned** — runs of blank lines collapsed, repeated spaces and tabs collapsed, per-line whitespace trimmed — and then checked against two bounds. Below 100 characters, the document is marked `skipped`: there is nothing to index. Above 50,000 characters, it is truncated.

What defeats extraction, reliably, is a **scanned PDF** — a document that is images of pages rather than text. Many older and locally produced agricultural documents are exactly this. The system currently records them as skipped with an explanation. Section 18.4 discusses optical character recognition as the fix.

5.5 Content classification

This is the step that produces most of the system's user-visible intelligence, and it is worth dwelling on.

Once a document's text is available, it is sent to the LLM — the first 4,000 characters of it — with a structured prompt asking for twelve specific pieces of information:

Field	What it captures
primary_topics	Topics from a controlled list (storage, drying, pest management, contamination, quality control, handling, marketing, …)
crops_mentioned	Crops from a controlled list (maize, beans, groundnuts, cassava, coffee, …)
regions	Ugandan districts and regions named in the text
document_type	Category — research paper, extension guide, policy document, statistical report, …
target_audience	Farmers, extension workers, researchers, traders, policymakers
language	The document's own language
relevance_score	0–10 for post-harvest knowledge base relevance
summary	Two to three sentences
key_practices	Specific practices or recommendations the document contains
actionable	Whether it contains practical, actionable advice
has_data	Whether it contains statistics, measurements or quantitative data
technical_level	Simple, intermediate, or advanced

The prompt supplies an explicit scoring rubric rather than leaving "relevance" to the model's judgement: 9–10 for dedicated post-harvest guidance, research or policy; 7–8 for substantial post-harvest content; 5–6 for passing mentions; 3–4 for agricultural but not post-harvest material; 0–2 for the unrelated. Anchoring a score to described bands makes it far more consistent across documents than an unanchored request for a number.

Constraining topics and crops to controlled vocabularies is equally deliberate. Left free, an LLM will produce "storage", "grain storage", "storage practices" and "post-harvest storage" across four documents that all mean the same thing — and the library's filters would fragment into near-duplicate categories. A fixed list makes the metadata aggregable.

The output is requested as JSON and parsed. The system normalises field-name variations the model produces (`primary_topics` versus `topics`), fills in defaults for anything missing,

and — if parsing fails entirely — falls back to a neutral default classification rather than failing the document.

Everything the public library displays comes from this step: the summary on each card, the topic, crop and region filters, the category label, the extracted practices, the actionability and data indicators. None of it was written by hand.

5.6 The relevance gate

The relevance threshold of **6.0** appears at three distinct points, and its consistency across them is intentional:

33. **URL triage.** URLs scoring below it are never downloaded.
34. **Content classification.** Documents whose full-text classification scores below it are stored, with their classification recorded, but marked `skipped` — no chunks are created, nothing is embedded, and the chatbot can never retrieve them. This matters: the document and the reason for its exclusion remain visible to an administrator, so the decision is auditable and reversible rather than a silent deletion.
35. **Public library visibility.** A document must score at or above it to appear in the library at all — and even then, only if an administrator has explicitly published it.

The effect is that the chatbot and the public library draw from the same trusted subset of the corpus by different paths, and no material enters either without clearing the same bar.

5.7 Chunking and embedding

Chunking follows the approach described in Section 2.4, with implementation details worth noting.

The splitter works on **paragraphs first**. Text is divided at blank lines, and paragraphs are accumulated into a chunk until adding the next would exceed the 1,000-token budget. If the text has no paragraph structure at all — common in text extracted from certain PDFs — it falls back to splitting on sentence boundaries. If a single paragraph is itself larger than the budget, that paragraph is split by sentences.

When a chunk is closed, the **overlap** is constructed by walking backwards through the paragraphs just emitted, keeping as many as fit within the 200-token overlap budget, and seeding the next chunk with them.

Token counts are estimated rather than computed exactly, using a four-characters-per-token heuristic. Exact tokenisation would require loading the specific tokeniser of the

specific model in use; the approximation is close enough for a size budget, and it keeps the chunking logic free of any dependency on which model is configured.

Each chunk is then **embedded**, in batches of 32, into a 1,536-number vector. Batching matters: embedding fifty chunks in two API calls rather than fifty is dramatically faster and cheaper, and the system preserves ordering when reassembling the batch results.

Every vector is stored with a **payload** — metadata carried alongside it in the vector database: the document identifier, the chunk's full text, the topics, crops and regions inherited from the document's classification, the source URL, the document type, the relevance score, and a timestamp. This payload is what allows a retrieval result to be immediately useful: the search returns not just "chunk 47 is relevant" but the actual text and its provenance, without a second database round-trip.

5.8 Storage and idempotency

Processing writes to three places:

- **MinIO** receives the original file, named by document identifier with its original extension.
- **PostgreSQL** receives one `content_chunks` row per chunk — its index within the document, its text, its estimated token count, its inherited metadata, and the identifier of its vector.
- **The vector database** receives the vector and its payload, keyed by that same identifier.

The link between the last two is essential and fragile. If a document is deleted, its vectors must be deleted too, or the chatbot will retrieve passages whose source no longer exists — citing a document the user cannot open. The deletion path in the administrative console is written with this specifically in mind (Section 12.3).

Content hashing provides idempotency. Every downloaded file's SHA-256 hash is computed and recorded. The same document reached by two different URLs, or uploaded manually after having been crawled, is recognised by its hash and reported as a duplicate rather than indexed twice. Without this, the same passage would appear repeatedly in retrieval results, crowding out other sources and distorting the evidence an answer is built from.

5.9 Manual ingestion

Not everything worth having is crawlable. A research paper handed over by a collaborator, a PDF behind a login, a document from a source with no configured crawler — all need a way in.

Manual ingestion accepts either an uploaded file (up to 50 MB) or a URL, from the administrative console or a command-line script that can also process an entire folder. It bypasses discovery and URL triage, but runs the **identical** downstream pipeline: extract, classify, apply the relevance gate, chunk, embed, store. A manually added document is not privileged; it must clear the same 6.0 relevance bar as anything crawled.

Manually uploaded files are anchored to a sentinel crawl run named `manual`, so the foreign-key relationships in the database remain intact and uploads are distinguishable from crawled material. Files uploaded rather than fetched are recorded with a synthetic `upload://` URL, which has an important downstream consequence: because the system does not host the file publicly, such a document cannot be published to the public library unless an administrator supplies a genuine public `http(s)` URL where the source can lawfully be read. The system will not become an unlicensed redistributor of other people's documents.

The ingestion result is explicit about outcome: `processed`, `duplicate`, `skipped` (with a reason) or `failed`, along with the document identifier, chunk count and relevance score — so the person uploading knows exactly what happened.

5.10 Scheduling and failure handling

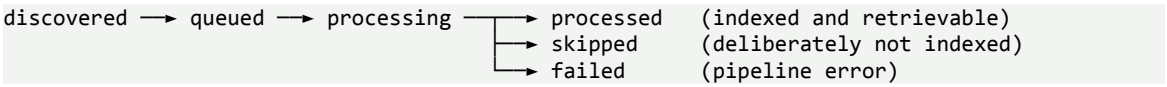
Three Prefect deployments run on schedules:

Schedule	Flow	What it does
Daily, 02:00 UTC	daily-scraper	Full pipeline: discovery across all enabled sites, then processing of up to 500 pending documents, then quality checks
Hourly	incremental-update	Lightweight discovery only — catches newly published material without a full crawl
Weekly, Sundays 03:00 UTC	reprocess-failed	Retries up to 100 documents that previously failed

The weekly retry deserves comment. Many failures are transient — a site was down, a request timed out, an API rate-limited. Rather than requiring someone to notice and intervene, the system simply tries again a week later. Failures that persist across several weeks are the ones worth human attention, and they are the ones that remain visible in the console.

Retries are configured per task at a granularity matched to what can go wrong. Downloads retry three times with 30-second delays, because network failures are common and usually brief. LLM classification retries once after 30 seconds. Whole discovery flows retry once after five minutes. Crawling a site retries twice.

The document status lifecycle is the pipeline's primary diagnostic surface:



Each transition is logged with an explanation written for a human reader rather than a developer — for example, that no usable text was extracted because the content may be empty, too short, unsupported, or image-only, and that consequently no chunks were indexed. The administrative console surfaces these explanations directly on each document, colour-coded by severity, with the underlying error detail attached where the outcome was negative. A deliberate refinement: the stored error text is only shown when it explains the *current* state, because a document that failed once and succeeded on retry would otherwise display a stale and misleading error.

The distinction between `skipped` and `failed` is meaningful and consistently applied. `Skipped` means the system worked correctly and decided not to index — the content was too short, or below the relevance bar. `Failed` means the system did not work. Conflating them would make the corpus statistics useless as a health signal.

6. The Retrieval and Answering Layer

This is the system's most visible component and the one where the retrieval-grounded commitment is enforced in practice.

6.1 The eleven steps of a chat turn

A single question passes through the following sequence. Each step is described with what it does, and what happens when it fails — because the failure behaviour is where most of the engineering actually lives.

1. Resolve the conversation session. The request carries a session identifier; the corresponding record is loaded. If it does not exist, the request is rejected with a clear error rather than silently creating a new conversation, because silently losing a user's history is worse than an error they can act on.

2. Load recent history. The last ten messages of the conversation are read from the database, newest-first, then reversed into chronological order.

3. Detect the language. If the user has explicitly selected a language, that choice wins outright. Otherwise the message is sent to Sunbird AI's language identification service. *If detection fails for any reason, the system defaults to English and proceeds* — a translation outage degrades the experience but never breaks it.

4. Translate to English. If the detected language is not English, the message is translated. *If translation fails, the original message is used unchanged.* A Luganda question searched as Luganda text will retrieve poorly, but poorly is better than not at all.

5. Embed and search. The English question is converted to a vector and the vector database is searched for the five closest passages.

5b. Apply the confidence gate. If no passages were returned, or if even the best similarity score is below 0.30, the system takes the refusal path (Section 6.4) and does not call the LLM at all.

6. Build conversation context. The last six messages — three exchanges — are formatted as prior turns for the model.

7. Synthesise the answer. The retrieved passages, the question, the history and the grounding instruction go to the LLM.

8. Translate the answer back. If the user's language is not English, the answer is translated. *If this fails, the English answer is returned* rather than an error.

9. Build sources and suggestions. Retrieved passages are grouped into document-level citations, and follow-up questions are assembled.

10. Persist everything. Both messages are written, along with the detected language, sources, suggestions, and — for non-English answers — the English original. The session's last-active timestamp is updated. In the *same transaction*, a background job is queued to score the answer later.

11. Return the response.

The pattern running through steps 3, 4 and 8 is the system's most consistent engineering principle: **every external dependency that is not strictly essential to producing an answer is wrapped so that its failure degrades the result rather than destroying it.** Translation down means English answers. Follow-up generation down means fewer suggestions. Catalogue lookup down means no related-question chips. The user still gets their answer.

6.2 Language detection and the English pivot

The pivot-through-English strategy described in Section 2.6 is implemented here. Its justification is concrete: the indexed corpus is in English, and the embedding model performs far better on English than on Uganda's low-resource languages. Embedding a Luganda question directly and searching an English index would place the query in a poorly-mapped region of the meaning space and return largely irrelevant passages.

The cost is that every non-English interaction incurs two translation round-trips, each introducing some loss of fidelity — a point Section 18.2 treats honestly.

One optimisation is worth noting because it is subtle. When an answer is translated, the system stores **both** versions: the translated answer that the user sees, and the English original in a separate column. When that message later becomes conversation history for a subsequent question, the *English* version is replayed to the model. This is not about fidelity but about cost: text in non-Latin or less-represented scripts tokenises far less efficiently, so replaying a Luganda answer as history can consume several times the tokens of the English equivalent, for no benefit — the model reasons over the history internally in whatever form it is given, and the user never sees it.

6.3 Semantic search in practice

The search retrieves the **top five** passages. That number is a balance: too few and the model lacks material to synthesise from; too many and the prompt fills with marginally relevant text that dilutes the good passages and increases cost.

A more interesting decision concerns **score filtering**, and the code documents it explicitly. An earlier version filtered individual chunks by their similarity score before synthesis, discarding any below a per-chunk cutoff. This was removed. The reason: for a question phrased colloquially or using synonyms, the genuinely relevant passages often score just below such a cutoff, because the embedding of a farmer's plain-language question is not tightly clustered with the embedding of formal technical prose even when they mean the same thing. Filtering per chunk was systematically discarding valid paraphrase and synonym matches, and it was doing so precisely for the users the system most needs to serve.

The replacement is a **whole-answer gate**: no chunk is filtered out, but the system checks whether the *best* score across the unfiltered top five clears 0.30. If it does, all five passages are used, including the weaker ones — because a passage that scores 0.34 alongside a passage that scores 0.61 usually genuinely contributes context. If even the best is below 0.30, the system concludes nothing relevant exists and refuses.

The 0.30 value is explicitly marked in the code as an interim, judgement-based figure pending calibration against real usage data. Every retrieval outcome and its top score is recorded as telemetry precisely so that this number can eventually be set from evidence rather than intuition (Section 9.6). Documenting a provisional threshold as provisional, in the code, is a small thing that materially improves the system's honesty about its own maturity.

6.4 The refusal path

When the confidence gate fails, the system:

- Returns a fixed message stating that it could not find relevant information in the knowledge base, and suggesting the user rephrase or browse the documents and catalogue.
- Records the outcome — `no_hits` or `below_min_top_score` — and the top score achieved, as telemetry, committed immediately so it survives any later failure.

- Still generates catalogue-based suggestions, because a question the system cannot answer from documents may well be answerable from a reviewed catalogue entry.
- Stores both messages so the refusal appears in history like any other turn.
- **Never calls the answering LLM.**

That last point is the important one. The cheapest, fastest thing the system could do at this moment is ask the model to answer anyway — and the model would produce something fluent and plausible. The whole design forbids it. A wrong drying threshold delivered confidently to a farmer is a worse outcome than an admission of ignorance, and the system is built to prefer the admission.

A secondary benefit is cost: refused questions consume an embedding call and nothing more.

6.5 The answer prompt

Two pieces of text govern what the model does.

The **system instruction** establishes role and constraint:

> *You are an agricultural knowledge assistant specialising in post-harvest practices in Uganda and East Africa. Answer questions using only the provided document excerpts. Always cite your sources by referring to the source number (e.g. [1], [2]). If the excerpts do not contain enough information to answer the question, say so clearly — do not fabricate information.*

Four instructions, each doing work: it scopes the domain, it forbids outside knowledge, it mandates citation, and it explicitly authorises admitting ignorance — which matters, because a model given only the first three instructions will strain to produce *something* rather than concede.

The **user prompt** supplies the evidence, with each passage numbered and labelled with its source URL, followed by the question and a request for a clear, structured answer with inline citations. Each passage is truncated to 300 characters in the prompt — a deliberate economy that keeps prompts small and cheap while still giving the model the substance of each source. The full text remains available in the retrieval payload for citation snippets and for faithfulness scoring.

Generation runs at **temperature 0.1**. Temperature controls randomness: at 1.0 a model varies its phrasing considerably between runs; at 0.1 it is nearly deterministic, choosing the most probable continuation almost every time. For creative writing, low temperature

is dull. For factual answering, it is exactly right — the same question should not produce materially different answers on successive attempts. Answers are capped at 1,024 tokens.

6.6 Conversation memory

Ten messages are loaded from the database; six are sent to the model. The asymmetry is intentional: the wider window is available for other uses (history display, ordering), while the narrower window is what the model sees.

Six messages is three question-answer exchanges. This is enough for the pronoun resolution that makes conversation natural — "how long will *that* last?", "what about *for beans*?" — without letting the prompt grow unbounded. Every message in the history is re-sent with every request, so an unbounded window means linearly growing cost and latency, and eventually a request the model rejects outright.

That rejection is handled explicitly. When a provider refuses a request as too large, the system raises a specific error and tells the user, in plain terms, that the conversation has grown too long and they should start a new chat or switch to a model with a larger context window. The alternative — a generic "internal server error" — would leave the user with no idea what to do.

6.7 Streaming responses

The web interface uses the streaming endpoint, which returns Server-Sent Events rather than a single response.

The stream carries typed events. **Status** events report the pipeline stage — `preparing`, `language`, `searching`, `reading_sources` (with a count of sources found), `synthesizing`, `followups`. **Token** events carry fragments of the answer as the model produces them. A final **done** event carries the complete answer, the full source list, the suggested questions and the detected language. An **error** event carries a failure.

The user-experience argument for streaming is well known but especially forceful here. A grounded answer takes several seconds: detection, translation, embedding, search, generation, translation back. On a slow mobile connection in rural Uganda it can take longer. A blank screen for eight seconds reads as a broken application; a screen that says "searching the knowledge base… found 5 sources… writing your answer" and then begins producing text reads as a system doing work. The perceived wait is dramatically shorter even though the actual wait is identical.

There is an architectural consequence. Once a streaming response has begun, its HTTP status code has already been sent — an error cannot be signalled as a normal HTTP failure. The system therefore performs **all authorisation and quota reservation before the stream is opened**, so that access failures surface as ordinary HTTP errors the client can handle cleanly, and only genuine mid-generation failures arrive as in-stream error events. The code comments this explicitly.

Streaming also deliberately does not retry. The rest of the LLM client retries failed calls with exponential backoff; the streaming path does not, because retrying a half-consumed stream would either duplicate text the user has already seen or silently drop part of the answer.

6.8 Citations as first-class objects

Retrieval returns passages. Users need documents. The gap between the two is bridged by a grouping step that materially improves the quality of the citation display.

For each retrieved passage, the system looks up its source document in the database and builds a reference containing the document's title, its public URL (preferring an administrator-supplied public source URL over the internal one), the site it came from, its highest similarity score across all its contributing passages, a text snippet of up to 320 characters, its AI-generated summary, its document category, up to six of its topics, and the list of citation numbers in the answer that refer to it.

When several retrieved passages come from the same document — which is common, since a relevant document usually contains several relevant passages — they are **merged into one reference** that accumulates all their citation numbers and keeps the best score. The user sees "this answer drew on three documents", with `[1]`, `[3]`, `[5]` attributed to the first of them, rather than five near-identical entries.

If the source document is published in the public library, the reference also carries an internal link to its library page, so a user can move straight from an answer to browsing the document behind it. If it is not published, no link is offered — the citation still shows what the answer was based on, but the system does not expose unpublished material through a side door.

There is a graceful fallback throughout: where a document record cannot be resolved, the reference is built from the vector payload instead, and where no title exists, one is derived from the source filename.

6.9 Suggested follow-up questions

After each answer, the system offers two to four next questions, assembled from two sources.

Catalogue suggestions come first. The question's vector is searched against the reviewed catalogue, and approved entries scoring above 0.65 are offered — already translated into the user's language. These are valuable because they are pre-vetted and free to answer: clicking one costs nothing.

Generated suggestions fill the remainder. The LLM is given the question, the answer and excerpts from the evidence, and asked for three concise standalone questions that "deepen, compare, quantify, or turn the answer into a practical next step" — with an explicit instruction not to imply the evidence contains information it does not. These are then translated into the user's language if needed.

The two sets are merged with case-insensitive deduplication, capped at four. The entire operation is wrapped so that failure produces fewer suggestions rather than a failed request.

6.10 Graceful degradation

Collecting the pattern in one place, because it is the system's defining reliability characteristic:

Failure	Consequence
Language detection unavailable	Defaults to English; user can override manually
Translation to English fails	Original message used for search; retrieval quality drops
Translation back fails	English answer returned
Follow-up generation fails	Fewer or no suggestions
Catalogue vector search fails	No related catalogue chips
Source document lookup fails	Citation built from vector payload instead
Vector database returns nothing	Honest refusal, no LLM call
LLM rejects the request as too long	Specific, actionable message to the user
Faithfulness scoring fails	Answer unaffected; job retried later

The only failures that produce an error rather than a degraded answer are the ones where no meaningful answer is possible: the session does not exist, the user is not authorised, quota is exhausted, or the answering model itself is unavailable.

7. Multilingual Support

7.1 The five Ugandan languages and why these five

Code	Language	Approximate reach
eng	English	Official language; medium of technical and government publication
lug	Luganda	Most widely spoken Ugandan language; Central region
ach	Acholi	Northern Uganda
nyn	Runyankole	Western Uganda
teo	Ateso	Eastern Uganda
lgg	Lugbara	North-western Uganda

The five were not chosen arbitrarily: they are the languages Sunbird AI supports, and between them they cover the major regions of the country, including the northern and eastern areas where post-harvest losses are most acute and English literacy lowest. Coverage is regional rather than exhaustive — Uganda has more than forty living languages — but it is the difference between a system usable by a minority and one usable across most of the country.

7.2 Sunbird AI as the translation backbone

Sunbird AI is a Ugandan organisation building language technology for Ugandan languages. Its API provides both translation and language identification for exactly this set of languages.

The choice was not a close call. Major commercial translation services support few or none of these languages, and where they do, quality is poor — these are low-resource languages, with little parallel text available for training, and a general-purpose model trained predominantly on European and East Asian language pairs simply has not learned them. A provider that has focused specifically on Ugandan languages, with access to locally collected data, is the only realistic option, and it also means the system's language capability is grounded in local rather than imported expertise.

The system uses three operations: detect the language of a text, translate into English, and translate from English.

7.3 Detection with a confidence threshold

Language detection returns a code and, when available, a confidence value. The system applies a threshold of **0.80**, but with a specific subtlety: the threshold is applied *only when the API actually returns a confidence value*. An earlier, naive implementation that treated a missing confidence as zero would have rejected every detection the API returned without one.

Detection is rejected — and English assumed — when confidence is below threshold, when the detected language is not one of the six supported, when it is already English (in which case no translation is needed), or when the call fails.

Defaulting to English on uncertainty is the right failure mode. A short message such as "maize storage" is genuinely ambiguous between several languages, and mistranslating an English question *out of* a language it was never in produces nonsense. English is the safe assumption because the corpus is English, and because the user can always override the detection explicitly in the interface.

7.4 Translating long text without destroying it

Translating a long answer is not simply a matter of sending it to the API. Two constraints collide.

First, there is a practical limit on how much text one request should carry; the system chunks at 500 characters and refuses inputs over 10,000 characters outright.

Second — and this is the interesting problem — **answers are Markdown**. They contain headings, bulleted lists, numbered steps, and sometimes tables. All of these depend on line structure: a list item is a list item because it starts on its own line with a marker. If a translation request merges two lines into one payload, there is no guarantee the response preserves the boundary, and a carefully structured answer can come back as an undifferentiated paragraph.

The system therefore **never sends a chunk spanning more than one original line**. Long text is split by line; blank lines are preserved untouched; each non-blank line short enough is translated on its own; and a single line still too long is split at sentence boundaries and rejoined with spaces. The translated lines are rejoined with newlines, and the document's structure survives intact. The code documents this reasoning explicitly, which is worth noting because it is exactly the kind of constraint that is invisible until it breaks and inexplicable afterwards.

7.5 Retries, backoff, and English fallback

Translation calls distinguish transient from permanent failures. HTTP 429 (rate limited), 503 and 504 (service unavailable or timing out), and network errors are treated as transient and retried up to three times with exponential backoff — one second, then two, then four. Other HTTP errors are permanent and raise immediately, with the full response logged for diagnosis.

Above this, every call site treats translation as optional. A translation outage means English answers reach users who asked in Luganda — a degraded experience, clearly, but a working one. The system also disables translation entirely and cleanly when no API key is configured, which is what makes English-only local development possible without any code changes.

7.6 Translation as a caching opportunity

Translation is the system's most repetitive cost. The same question, asked by twenty different users, is translated twenty times, and its answer translated back twenty times.

The Q&A catalogue attacks this directly. When an entry is approved, it is translated **once** into all five languages — question and answer — and those translations are stored permanently. Every subsequent user who matches that entry receives it in their language with **no translation call at all**, and no LLM call either.

This is one of several places where the catalogue is not merely a convenience feature but a structural cost optimisation, and Section 8 develops the argument.

8. Browse-First Design — Library, Catalogue, and the Ask Gate

This is the system's most distinctive product decision, and the one that most clearly separates it from a conventional "chatbot over documents" build.

8.1 Why chat was demoted from the homepage

The system originally opened onto a chat interface. Every question — including questions asked and answered many times before, and questions whose answer is simply "read this document" — ran the complete pipeline: language detection, translation, embedding, vector search, LLM synthesis, translation back.

Three problems with that, in ascending order of seriousness.

It is expensive. Every one of those steps costs money. An answer to a question already answered last week costs exactly as much as the first time.

It is slow. Several seconds, plus network latency, for information the system already possessed in finished form.

It is unnecessary, given the shape of this corpus. This is the argument that actually decided it. The knowledge base is *finite and well-classified*. Every document already carries a summary, a topic list, a crop list, a region list, a category, and a set of extracted key practices — all generated during ingestion, all sitting unused in the database. A large fraction of user need is "show me what exists about drying maize", which is a *browsing* need, and answering it by generating text from a language model is using an expensive, non-deterministic tool for a job that a filtered list does better, faster, free, and with complete transparency.

The redesign made browsing the default and chat the fallback. Two systems that cost nothing to serve sit in front, and chat runs only when neither has the answer — where it also, off its own critical path, produces the material that grows the catalogue.

8.2 The public document library

The library is generated entirely from classification metadata that already exists. No LLM is invoked to serve any part of it.

Visibility. Every library query applies the same four-part rule, and all four conditions must hold:

36. The document's status is `processed`.
37. Its classification relevance score is at or above 6.0.
38. Its `publicly_visible` flag has been set by an administrator.
39. If it was uploaded rather than crawled, it carries a genuine public `http(s)` source URL.

The third condition is the important one. Being indexed does not make a document public; **relevance alone never grants visibility**. A human decides. This separation means the chatbot can draw on material that is not published — for instance, a document whose licensing permits internal use but not redistribution — while the library only ever exposes what someone has affirmatively approved.

The fourth condition prevents the system from becoming an unlicensed document host. An uploaded PDF has no public URL of its own; publishing it without a lawful external link would mean presenting someone else's document as if the system were entitled to distribute it.

Faceted browsing. The library offers filters over topics, crops and regions, each with counts. These are computed by expanding the array columns in PostgreSQL and grouping, scoped by the visibility rule so counts never reveal unpublished material. The result is cached in memory for five minutes, because facet values only change when a crawl finishes processing new documents, and recomputing them on every page load would be wasteful.

Listing and detail. The listing supports filtering by topic, crop and region, a text search over titles and summaries, and pagination. The detail view adds key practices, the actionability and data indicators, and **up to three excerpts of up to 800 characters each** — deliberately bounded. The system shows enough for a reader to judge whether the document is worth pursuing, then sends them to the source. It is a discovery surface, not a mirror of other people's content.

Uniform 404s. A request for a document that does not exist, is unprocessed, is below threshold, or is unpublished returns exactly the same "not found" response. No distinction is made, so nothing about the existence or state of unpublished material can be inferred by probing.

8.3 The Q&A catalogue

The catalogue is a living FAQ that the system builds from its own use. Each entry holds:

- The question, in English, rewritten into a clear standalone form
- The answer, in English
- Translations of both into all five Ugandan languages
- The embedding of the question, for semantic matching
- The source documents cited by the original answer
- Its automated grounding score and curation score
- Counts of how often it has been served and how often marked helpful
- A count of how many later candidates turned out to duplicate it
- Its status in the review lifecycle

The status lifecycle is:

```
pending ──► publishing ──► approved ──► retracted
   │              │
   │              └──► publish_failed ──► (retry) ──► publishing
   └──► rejected
```

`Pending` means awaiting human review. `Publishing` means approved and undergoing translation and indexing. `Approved` means live and servable. `Publish_failed` means an error occurred during publication, with the error text retained and a retry available. `Rejected` and `retracted` mean a human declined it, before or after publication respectively.

8.4 How an answer becomes a catalogue entry

The candidate pipeline runs entirely in the background worker, and each stage can eliminate a candidate.

Stage 1 — Enqueue. When a chat answer is stored, a job is written *in the same database transaction*, carrying the message identifier and the identifiers of the passages that fed the answer. Because it is one transaction, the job cannot be lost if the process dies immediately afterwards, and it cannot exist for a message that was never stored.

Stage 2 — Faithfulness scoring. The worker retrieves the answer and its source passages and computes a BERTScore (Section 9.2), measuring how well the answer's content is actually supported by the sources. The score is stored on the message regardless of what happens next, because it feeds the quality dashboard. **If the F1 score is below 0.60, the candidate is eliminated.** An answer that does not stay close to its evidence is not a candidate for permanent publication, whatever else may be true of it.

Stage 3 — Curation scoring and question rewriting. The LLM is asked to assess whether the exchange is useful as a reusable public entry, returning a score from 0 to 1 and a rewritten, standalone version of the question capped at 300 characters. The prompt explicitly instructs it to prefer practical, generally reusable questions and to reject greetings, personal requests, vague follow-ups, and answers that depend on missing conversational context.

The rewriting matters as much as the scoring. A real user asks "ok but what about beans?" — meaningless out of context. The rewrite turns it into "How should beans be dried before storage to prevent spoilage?", which is a question a future user might actually ask and which embeds into a meaningful position in the semantic space.

Stage 4 — Duplicate detection. The rewritten question is embedded and compared against every existing pending, publishing or approved entry. If the closest match scores **0.90 or above**, the candidate is not created; instead the existing entry's duplicate counter is incremented. That counter is itself useful signal: an entry duplicated thirty times is demonstrably a frequently asked question.

Stage 5 — Creation. Surviving candidates are stored as `pending` with their scores, source message and source passages recorded, awaiting review.

A deliberate asymmetry governs stage 3. In **manual mode** — the default — a low curation score does *not* eliminate a candidate. It is recorded and shown to the human reviewer as evidence, and the human decides. In **automatic mode**, where no human sees it, the 0.70 minimum is enforced strictly. The code comments the reasoning: a hidden rejection gate in front of a human reviewer removes information the reviewer needs, whereas automatic publication has no safety net and must enforce it.

8.5 Human review, publication, and retraction

The administrative console presents the pending queue with, for each candidate, the rewritten question, the answer, the cited sources, the grounding score, the curation score, the duplicate count, and whether translations are complete.

Approve queues a publication job. The worker then translates the question and answer into all five languages — committing after each so a failure part-way through does not lose completed work — verifies that every required translation is present and that the embedding is valid, writes the entry's vector into the catalogue's own vector collection, and marks it approved. From that moment it can be matched and served.

Reject marks it rejected. **Retract** removes an approved entry from service and queues a job to delete its vector, so it stops matching. **Retry** re-attempts a failed publication.

Every approval and rejection is recorded as telemetry along with the candidate's two scores. This creates exactly the dataset needed to answer the question "do the automated scores predict human judgement?" — and therefore whether automatic publication could ever be justified (Section 9.7).

8.6 The ask gate

The gate is the mechanism that puts the catalogue in front of chat.

Every question typed on the homepage goes first to a matching endpoint. The question is language-detected, translated to English if needed, embedded, and searched against the catalogue's vector collection. If an approved entry scores **0.82 or above**, it is returned directly, in the user's language, with its citations, and its served count is incremented.

The 0.82 threshold is a deliberately high bar. Serving a stored answer to a question it does not actually address is a worse failure than falling through to live chat, because the user has no way to know the answer was written for a different question. The threshold is set to prefer false negatives — falling through unnecessarily — over false positives.

Notably, the matching endpoint is **public**: no access code is required. It uses language detection, translation, embedding and vector search, none of which involve an LLM, so it is cheap. It does carry a per-IP rate limit of 30 requests an hour, because it is not free.

On a miss, the question is stored in the browser and the user is passed to live chat — where, if they do not yet hold an access code, they are asked for one, and their question is preserved across the redemption so nothing is retyped.

8.7 Feedback loops

Three signals are collected:

"This answered it" increments the entry's helpful count and records positive feedback.

"This doesn't answer my question" records a rejected match — evidence that the 0.82 threshold admitted something it should not have — and immediately hands the user to live chat. This telemetry is explicitly non-blocking: if recording it fails, the handoff proceeds anyway, because feedback collection must never obstruct the user's actual goal.

Served counts accumulate on every match, showing which entries are carrying real load.

Together with the recorded match scores of both hits and misses, this produces the evidence base for tuning the thresholds — the difference between "0.82 feels right" and "0.82 is where the data says the boundary is".

8.8 What this achieves

Cost. A catalogue hit consumes one embedding call. A live chat turn consumes language detection, two translations, an embedding, an LLM synthesis, a follow-up generation, and later a faithfulness scoring pass and a curation call. The ratio is not marginal.

Latency. Effectively instant versus several seconds.

Trust. A catalogue answer has passed an automated faithfulness check *and* a human review. A live answer has passed neither at the moment it is delivered.

Access. Browsing the library, browsing the catalogue, matching a question against the catalogue, and reading a matched answer are all available with **no access code at all**. The gate applies only when a request will actually enter live chat and consume metered capacity. This is what makes the access-control model defensible: the system is not paywalled, it is metered at exactly the point where it costs money.

Improvement over time. Every live conversation is a candidate contribution to the catalogue. The more the system is used, the more questions it can answer for free.

9. Measuring Whether the Answers Are Any Good

9.1 The evaluation problem

Building a system that produces confident answers is straightforward. Knowing whether those answers are any good is not.

The obvious approach — asking the LLM to rate its own answer — is close to worthless. A model that was confident enough to produce a hallucinated claim will be confident enough to rate it highly. Self-assessment shares the failure mode it is meant to detect.

Human review is the gold standard but does not scale. Nobody is going to read every answer.

What is needed is an **independent, automated signal**: a measurement produced by a different mechanism, that specifically detects the failure mode that matters most here — an answer drifting away from its sources.

9.2 BERTScore explained plainly

BERTScore is that signal. It compares a generated answer against the source passages it was supposed to be based on, and returns a number reflecting how much of the answer's meaning is actually present in those sources.

The plain-language version: it checks whether the answer is *saying things the sources said*.

The mechanism, without unnecessary machinery: an older but still highly capable class of language model (BERT and its relatives) can produce a vector for each token of a text that reflects that token's meaning **in its context** — so "store" in "store the grain" and "store" in "the village store" receive different vectors. BERTScore embeds every token of the answer and every token of the sources this way, then matches each answer token to its best-matching source token and averages the similarities.

This is fundamentally better than word-overlap measures for this purpose. An answer that says "keep the grain dry before sealing it" is well supported by a source that says "reduce moisture content prior to hermetic storage", despite sharing no words — and word-overlap scoring would call it unsupported. Conversely, an answer that introduces a specific number or a specific chemical name absent from the sources will score poorly, because those tokens have no good match anywhere in the references. That is exactly the failure the system needs to catch.

Three numbers are produced:

- **Precision** — how much of the *answer* is supported by the sources. **This is the hallucination detector.** Low precision means the answer contains material the sources do not support.
- **Recall** — how much of the *sources* is reflected in the answer. Low recall is not a defect; a good answer is selective.
- **F1** — the harmonic mean of the two, used as the single summary figure.

The system uses F1, with a threshold of **0.60** for catalogue candidacy.

An important limitation, stated plainly: BERTScore measures **faithfulness to sources, not correctness**. An answer that accurately reflects a source document that is itself wrong will score highly. The measure guards against the system inventing things. It does not guard against the corpus being wrong, and nothing in the system does — that is a curation responsibility.

9.3 How and where it runs

BERTScore uses `distilbert-base-uncased`, a compressed variant of BERT chosen because it runs acceptably on CPU. Scoring runs on **CPU explicitly**, so the system needs no GPU. Source passages are joined and truncated to 3,000 characters, comfortably inside the model's 512-token limit.

Crucially, **scoring never happens during a user's request**. It runs in the background worker, triggered by the job queued alongside the answer. This is a change from the original design, which used a web-framework background task inside the API process; moving it to a separate worker keeps the machine-learning model — and its memory footprint — out of the API entirely, and makes scoring durable across restarts.

The model is loaded once as a thread-safe singleton and reused for every subsequent call, since loading it per invocation would dominate the cost.

The score is written to the message record as structured data: precision, recall, F1, the model used, and a timestamp. Recording the model name matters — scores from different models are not comparable, and without it a future model change would silently corrupt the historical series.

9.4 The Chat Quality dashboard

The console exposes this data to administrators.

Aggregate statistics report how many answers have been scored, how many are unscored, the mean F1 across all scored answers, and the distribution across four bands: below 0.4, 0.4–0.6, 0.6–0.8, and 0.8–1.0. These are computed in a single database query over the JSON score column.

The message list shows scored answers paginated and filterable by score range, each alongside the question that produced it. This is the practically useful view: filter to answers scoring below 0.4, read them next to their questions, and see what is going wrong. Typically it is a question the corpus does not really cover, where retrieval returned tangentially related passages and the model stretched them further than it should have.

Attaching each answer to its preceding question is done with a database-side lateral join — for each assistant message, find the most recent user message before it in the same session — which is far more efficient than fetching messages and pairing them in application code.

9.5 Operational telemetry

Beyond answer quality, the system records operational metrics for six categories of event: chat retrieval outcomes and their top scores, catalogue match attempts and their scores, catalogue candidate outcomes, reviewer decisions, background worker outcomes, and user feedback.

The rule governing what may be recorded is stated explicitly in the code and enforced by convention throughout:

> *Metadata must remain operational: never include questions, answers, tokens, phone numbers, IP addresses, access codes, or exception messages.*

So the system records that a retrieval was `synthesized` with a top score of 0.58 — not what was asked. It records that a catalogue match failed at 0.71 — not the question that missed. It records that a job failed with error type `TranslationError` — not the error message, which might quote user content.

This is deliberately restrictive, and it constrains debugging: an operator can see *that* something is going wrong and how often, but not the specific content involved. That is the correct trade-off for a system handling agricultural questions from identifiable individuals over WhatsApp, and it means the metrics table can be queried, exported and analysed without any privacy review.

Retrieval telemetry is committed **immediately**, before synthesis begins, so that a subsequent LLM failure does not erase the record that retrieval succeeded — otherwise the metrics would systematically under-count exactly the interactions most worth investigating.

9.6 The calibration report

The system's thresholds — 0.30 for retrieval confidence, 0.82 for catalogue matching, 0.60 for faithfulness, 0.70 for curation, 0.90 for duplication — were initially set by judgement. The calibration report exists to replace judgement with evidence.

An administrative endpoint summarises a configurable window (30 days by default) and returns:

- **Current configuration** — every threshold as presently set, so the report is self-describing.
- **Chat retrieval** — counts by outcome (`synthesized`, `no_hits`, `below_min_top_score`) and the distribution of top scores overall and per outcome, reported as minimum, median, 90th percentile and maximum.
- **Catalogue matching** — attempts, matches, match rate, and score distributions separately for hits and misses. The gap between these two distributions is what tells an operator whether 0.82 sits in the right place.
- **Catalogue candidates** — outcome counts and the distributions of grounding and curation scores.
- **Reviewer outcomes** — approvals, rejections, approval rate, and the score distributions of approved versus rejected candidates. If approved and rejected candidates show clearly separated score distributions, the automated scores are predictive; if they overlap heavily, they are not, and no threshold on them would substitute for human review.
- **Worker health** — job counts by status, permanent failures, and attempt distributions.

The report's most important property is stated in its own code comment: it **summarises evidence without changing any runtime threshold**. It is an instrument, not a controller. Nothing about the system adapts automatically in response to it. A human reads it and decides.

9.7 Evidence gates for automation

The report includes an explicit assessment of whether automatic catalogue publication could be considered at all. Three conditions must hold simultaneously:

40. At least **50 human-reviewed samples** exist in the window.
41. The background worker has recorded **zero permanent failures**.
42. Both of the above, evaluated together.

And the response carries a final field: `automatically_changes_mode: false`.

This is a small piece of engineering with a large amount of judgement behind it. The system computes whether it *could* be trusted to publish without human review, reports that assessment, and then does nothing about it — because the decision to remove a human from a quality-control loop is a human decision, made deliberately, not one a system should make about itself on the basis of its own metrics.

10. Access Control, Quotas, and Cost Governance

10.1 The problem

Every live chat message costs real money. An answer consumes: a language detection call, up to two translation calls, an embedding call, an LLM synthesis of up to 1,024 tokens with several passages of context, a follow-up generation call, and — asynchronously — a faithfulness scoring pass and a curation call. Multiply by the volume an openly accessible endpoint attracts, including automated traffic that is not asking about maize storage at all, and an unbounded chat endpoint is an unbounded liability.

The original system had only per-IP and per-phone hourly rate limits. Those bound the *rate* of abuse, not its total. There was no concept of an authorised user.

At the same time, the system's purpose is to make post-harvest knowledge *more* accessible, so a blanket paywall is self-defeating. The design resolves this by metering precisely and only where cost is incurred: **browsing is free and open; live chat is metered.**

10.2 Three-word access codes

An access code is three ordinary words joined by hyphens — `mango-river-granary` — generated from a curated wordlist using a cryptographically secure random source.

The format was chosen against the realities of the target users. It is typeable on a low-end phone keypad without a symbol keyboard. It survives being read aloud over a phone call or written on a whiteboard at a training session. It contains no ambiguous characters — no confusion between `0` and `O`, or `1` and `l`. And it can be sent as an ordinary WhatsApp message, which is precisely how WhatsApp users redeem it.

Codes are **shared, not personal**. One code is issued to an organisation — "NARO Extension Officers, Q3 batch" — with a monthly message allowance covering the whole group. This matches how the system is actually deployed: through partner organisations, cooperatives and extension programmes, not through individual registration. It also avoids requiring personal accounts, personal data, or a registration flow from users who may have neither email nor reliable literacy.

Each code carries a label, an optional organisation name, an optional contact message shown to users, a monthly message limit, a status (active or revoked), a usage count, and the start of its current billing period.

10.3 Codes, grants, and identity binding

A code by itself grants nothing. Redemption creates a **grant** — a durable binding between the code and a specific identity.

On the web, redemption returns a freshly generated 256-bit random token, stored in the browser. The grant records only the token's hash. The plaintext token exists exactly once, in the response to the redemption request; the server cannot recover it afterwards.

On WhatsApp, the identity is the phone number, and the grant records only its keyed hash.

A database constraint enforces that any given identity has **at most one active grant** at a time. Where a phone number redeems a new code while already holding a grant — the normal path when a code has been revoked and a replacement issued — the existing grant is marked `replaced` and the new one created **in the same transaction**, so there is never a moment where the number has two active grants or none.

Grants are permanent by design. There is no session expiry, no periodic re-authentication, no login. A farmer who redeems a code once continues to work indefinitely until an administrator revokes the underlying code. Requiring periodic re-entry of a code would be a significant usability tax on exactly the users least able to absorb it.

10.4 Quota enforcement

Two independent limits apply to every live message.

The per-code monthly limit is set when the code is issued and adjustable afterwards.

The global monthly ceiling — 100,000 messages by default — applies across all codes combined. It is a circuit breaker: even if every code's individual limit is generous, the system cannot exceed a total the operator has budgeted for. This protects against the case where an administrator issues more capacity than the operator can actually fund.

Atomic reservation is where this gets technically interesting. Consider a group with one message remaining and two users sending simultaneously. A naive implementation reads the counter (both see 99 of 100), checks it (both pass), and increments it (both write 100) — and two messages are served against one remaining unit. At small scale this is a rounding error; as a pattern it is a correctness bug that undermines the entire limit.

The system prevents it by performing the whole reservation inside a single database transaction with **row-level locks**. It locks the global counter row, then the code's row, and — critically — **always in that order**. Two transactions that acquire the same two locks in opposite orders can deadlock, each holding what the other needs. A fixed global-then-code ordering makes that impossible.

Within the lock, the system performs the **lazy monthly reset**: if the recorded period start is in an earlier calendar month than now, the counter is zeroed and the period start moved forward, before any check is applied. This means no scheduled job is needed to reset quotas at month boundaries — the reset happens naturally on the first use of the new month. Fewer moving parts, no midnight cron to fail silently.

It then checks that the code is not revoked, that the code's usage is below its limit, and that the global usage is below the ceiling — rolling back and raising a specific error at the first failure — and only then increments both counters and commits.

The whole reservation completes **before any expensive work begins**. A user over quota costs a database transaction, not an LLM call.

10.5 Security measures

Keyed hashing with domain separation. Access codes and phone numbers are stored as HMAC-SHA-256 digests computed with a server-side secret, never in plaintext. Two properties follow. First, an attacker with a copy of the database cannot brute-force a three-word code or a phone number without also having the secret, which a simple unkeyed hash would not prevent given how small those input spaces are. Second, the *domain* — a fixed string identifying what is being hashed — is mixed into the input, so the hash of a phone number in the grant context differs from the hash of the same number in the throttling context. Digests cannot be correlated across purposes.

Unkeyed hashing where appropriate. Web tokens are hashed with plain SHA-256, and the difference is principled: a 256-bit random token has enough entropy that brute-forcing it is infeasible regardless, so keying adds nothing.

Redemption throttling. Repeated guessing of a three-word code is the obvious attack. Failed redemption attempts are recorded against a hash of the requester's identity — IP address on the web, phone number on WhatsApp — and a sliding one-hour window permits at most five. A successful redemption clears the window.

Log redaction. WhatsApp messages that *look like* access codes are logged as `[redacted access code]` rather than in full. This check is computed before any database access, specifically so the log line can be written safely. Without it, valid access codes would sit in plaintext in application logs, which are typically far less protected than the database.

Timing-safe comparison. Administrator credentials are compared with a constant-time function, so response timing cannot reveal how much of a guessed username or password was correct.

10.6 The layered defence

Access control did not replace the existing protections; it was added in front of them.

Layer	Protects against	Scope
Webhook signature verification	Forged WhatsApp requests	Per request
Message deduplication	Meta's delivery retries causing duplicate processing	Per message
Per-IP / per-phone rate limits	Burst abuse	Per hour
Access grant	**Unauthorised use**	**Per identity**
Per-code quota	**Budget overrun by one group**	**Per month**
Global ceiling	**Total budget overrun**	**Per month**

Each layer catches what the others cannot. Rate limits bound bursts but not totals; quotas bound totals but not bursts; grants bound who, not how much. Together they bound all three.

10.7 Failing usefully

The system defines a stable set of error codes, each with an appropriate HTTP status, so that clients can respond specifically rather than generically:

Code	Status	Meaning
access_required	401	No token supplied
access_invalid	401	Token or code not recognised
access_revoked	403	The group's code has been revoked
session_forbidden	403	This session belongs to a different grant
quota_exhausted	429	The group's monthly messages are used
global_limit_reached	503	System-wide capacity reached
redemption_rate_limited	429	Too many failed code attempts

Quota errors additionally carry the timestamp at which the period resets, so the interface can tell the user precisely when they will be able to continue rather than offering a vague "try again later".

The most considered behaviour here is what happens when a group's quota is exhausted. The user is not logged out and their history is not hidden. They can still read every past conversation, still browse the library, still browse and match against the catalogue. Only the ability to send a *new* live message is suspended, with a message explaining when it returns. The interface renders this as a read-only conversation with an explanatory strip rather than a locked door.

Similarly, a revoked code produces a message explaining that access has been paused and can be reactivated by its administrator, or replaced with a new code — not a bare "forbidden".

10.8 One genuinely hard edge case

Consider a WhatsApp user who has already redeemed a code and who sends the message: **"dry my maize"**.

Normalised, that is three lowercase words joined by hyphens: `dry-my-maize`. It is indistinguishable in shape from an access code. Treated as one, it fails to match any real code, is recorded as a failed redemption attempt, counts against the user's five-per-hour throttle, and produces the reply "that code was not recognised" — to a user who was asking a perfectly ordinary question and now believes the system is broken.

The resolution is a careful piece of case analysis. When a code-shaped message fails to match a real code:

- **If the sender has no existing grant**, they were almost certainly attempting to redeem. Record the failed attempt and reply that the code was not recognised.
- **If the sender already has an active grant**, they were almost certainly asking a question that happens to have three words. Do *not* record a failed attempt. Fall through to the normal chat path and answer it.

And a further distinction: if the message matches a real code that has been *revoked*, that is a genuinely different situation — the user is trying to use a code that exists but is no longer valid — and it retains the explicit revoked-code handling regardless of grant status.

This is a small amount of logic protecting a specific and entirely predictable user experience failure, and it is documented at length in the code because the reasoning is not self-evident from reading it.

11. The Two Channels

11.1 The web application

The web application is a **single-page application** built with React, bundled by Vite, and styled with Tailwind CSS. "Single-page" means the browser loads the application once, and subsequent navigation between library, catalogue, chat and admin happens without a full page reload — the application updates the view in place and fetches only the data it needs.

Route structure:

Route	Purpose	Access
/	Ask gate — the question box and entry point	Public
/chat	Live chat interface	Access grant required
/documents, /documents/:id	Public document library	Public
/catalogue, /catalogue/:id	Reviewed Q&A catalogue	Public
/admin/login	Administrator login	Public
/admin/*	Administrative console (nine pages)	Admin JWT required

The public routes and the console live in the same application and are served by the same static bundle. Administrative routes are wrapped in a guard that redirects to the login page when no valid token is held. This is a convenience boundary, not a security boundary — the actual protection is server-side, where every administrative endpoint independently verifies the token. A user who navigated directly to an admin route would see the interface shell and receive 401 responses for every request it made.

State management uses focused custom hooks rather than a global state library. `useSession` handles creating and restoring a chat session. `useChat` handles messages, streaming, history loading, language state and model preference. An access context handles the access token, its status, and redemption. Each owns one concern, and components consume what they need.

Rendering answers safely. Answers are Markdown, rendered with `react-markdown`. Rendering model-generated content into a page is a potential injection vector, so links are passed through a shared safety helper that permits only `http` and `https` URLs; anything else — `javascript:`, `data:`, malformed strings — is rendered as inert plain text rather than a clickable anchor. The same helper guards the library's "view original source" links.

Visual design. The interface uses a dark, muted palette built around deep greens and warm golds, with a serif display face for headings and a monospace face for small labels. It reads as a research instrument rather than a consumer chat product, which is appropriate to the audience and to the system's emphasis on evidence and provenance.

Progress display. The `ResearchProgress` component maps each streamed status event to a human-readable stage with an explanatory line — "Searching the knowledge library / Looking for the strongest evidence passages", "Reviewing the retrieved evidence / Checking which passages can support the answer", and when the source count is known, "Reviewing 5 relevant passages for support". The stages are presented in order with the current one highlighted. The effect is that the user watches the retrieval-grounded process happen, which is itself a small argument for the system's trustworthiness.

Sources panel. Citations open in a side panel showing each contributing document with its title, site, score, summary, topics, snippet, citation numbers, and links to both the original source and — where published — the library page.

11.2 Web session and identity model

The flow across the ask gate and chat is more carefully constructed than it appears.

A user types a question on the homepage. The question is matched against the catalogue. On a miss — or when the user asks a follow-up to a match — the question is written to the browser's local storage and the access gate is presented. When the user redeems a code (or if they already hold a token), a session is created, the pending question is moved to a handoff key, and the browser navigates to the chat view, which reads the handoff and sends the question automatically.

The consequence is that **a user never retypes their question**, even though they crossed an authentication boundary in the middle of asking it. If they close the browser at the gate and return later, the pending question is still there.

Session ownership is enforced server-side on every request. A session belongs to the access grant that created it, and any attempt to read or post to a session belonging to a different grant is refused. Possessing a session identifier is not sufficient; the request must also carry the token of the grant that owns it.

The thread history panel lists the caller's sessions, titled by their opening question (truncated to 80 characters), with message counts and timestamps, ordered by last activity.

Only sessions belonging to the caller's grant are listed, and only those that actually contain messages — empty sessions created and abandoned do not clutter the list.

11.3 WhatsApp integration

WhatsApp is reached through Meta's Cloud API, which delivers inbound messages to the system as webhooks and accepts outbound replies over HTTP.

Verification handshake. When the webhook is first registered, Meta issues a GET request carrying a mode, a verify token and a challenge value. The system checks the mode and compares the token against its configured value, and on success returns the challenge as a plain integer. This proves to Meta that the endpoint is controlled by someone who knows the shared secret.

Signature verification. Every inbound POST carries a header containing an HMAC-SHA-256 signature of the raw request body, computed with the application secret. The system recomputes it and compares using a timing-safe comparison. Without this, anyone who discovered the webhook URL could inject arbitrary messages, consuming quota and polluting the conversation record. If no application secret is configured, the system logs a prominent warning that verification is disabled — a deployment misconfiguration is made loud rather than silent.

Deduplication. Meta retries deliveries it believes failed, including cases where the system replied successfully but slowly. Without deduplication a user could receive the same answer twice and be charged twice against quota. The system keeps a bounded, time-limited record of recently seen message identifiers — capped at 1,000 entries with a one-hour expiry, oldest evicted first — and silently acknowledges repeats.

Rate limiting. A sliding one-hour window per phone number, defaulting to 30 messages.

Non-text messages. Images, voice notes, documents and location shares are answered with an explanation that only text questions can be handled. This matters: silence would read as a broken service, and voice notes in particular are a common way for users with limited literacy to communicate. Section 18.4 identifies voice-note transcription as a high-value future capability.

The 15-second budget. Meta requires a response within 20 seconds. The system runs the retrieval pipeline with a 15-second timeout, leaving margin. On timeout the user receives an apology and an invitation to retry, rather than nothing. The pipeline itself runs in a worker thread so the timeout can be enforced without blocking the server's event loop.

Conversational flow. The complete state machine for an inbound message:

```
verify signature → deduplicate → rate-limit → resolve grant
      │
      ├─ message looks like a code? → redeem (with the Section 10.8 disambiguation)
      ├─ no grant, ordinary message? → ask for an access code
      └─ has grant → check code active → reserve quota → find/create session
                                                              │
                                                  run RAG (15s) → reply
```

Every reply attempt is individually wrapped so that a failure to send one message — for example, a rate-limit response from Meta — never propagates as an unhandled error that would cause Meta to retry the whole delivery.

11.4 What differs between channels and why

Capability	Web	WhatsApp
Streaming answers	Yes	No — one message
Source citations	Full panel with scores, summaries, links	Not shown
Thread history	Browsable and resumable	Implicit, one continuous conversation
Model selection	User-selectable	Server default
Suggested follow-ups	Shown as clickable chips	Not shown
Language override	Picker in header	Automatic detection only
Library and catalogue browsing	Yes	No
Identity	Token in browser storage	Phone number

The differences follow from the medium. WhatsApp offers a plain text message and nothing else — no panels, no chips, no interface controls — so the WhatsApp experience is deliberately minimal rather than a degraded copy of the web one. The trade-off is real: WhatsApp users receive answers without visible citations, which is a genuine weakening of the system's transparency commitment for the audience least able to verify claims independently. It is a limitation the medium imposes rather than one the design chose, and it is recorded as such in Section 18.2.

12. The Administration Console

Nine pages, all behind a single authentication boundary.

12.1 Access and authentication

Administrators log in with a username and password held in configuration. On success the server issues a **JWT valid for eight hours**, which the browser stores in session storage — cleared when the browser closes — and presents with every subsequent request. Every administrative endpoint independently verifies the token's signature and expiry; expired and invalid tokens are distinguished so the interface can say "your session expired, log in again" rather than "invalid token".

Credential comparison is timing-safe. A single shared administrative credential is a deliberate simplification appropriate to the deployment's scale; Section 18.2 notes it as such.

12.2 Overview

The landing page reports total documents discovered, documents successfully processed, the processing rate as a proportion, the number of vectors in the index, and a chart of documents discovered per day over the last 30 days. Together these answer "is the pipeline working?" at a glance: a flat activity chart means crawling has stopped; a falling processing rate means documents are failing.

12.3 Documents

A paginated, filterable list — by source site, by status, sorted by discovery date or relevance — showing for each document its title, URL, site, type, status, relevance score, discovery date, publication state, summary, category, key practices, topics, crops and regions.

Each row carries a **processing explanation** in plain language with a severity level: processed documents report that extracted content was indexed; skipped ones that processing was intentionally skipped and no content indexed; failed ones that the document was not fully indexed, with the underlying error attached. As noted in Section 5.10, the error detail is shown only when it explains the current state, so a stale error from a since-succeeded retry is not displayed.

Publication toggles a document's visibility in the public library. Two guards apply: only `processed` documents can be published, and an uploaded document requires a valid public `http(s)` source URL first. Publication records who published it and when.

Renaming is available because crawler-derived titles, generated from URL path segments, are frequently poor.

Deletion is the operation with the most care in it, because it must not leave the system inconsistent. The order is deliberate:

43. Verify the document is in a deletable state — `processed`, `failed` or `skipped`. A document mid-ingestion cannot be deleted, since concurrent processing would recreate rows behind the deletion.
44. **Delete its vectors from the vector database first.** If this fails, the operation aborts with an explicit message that no database records were deleted. This ordering is the crux: had the database rows been deleted first and the vector deletion then failed, the index would retain passages pointing at a document that no longer exists — and the chatbot would cite a document nobody can open. Better to leave the document fully present and report the failure.
45. Delete its queue entries, classification, and chunks, then the document itself, in one transaction.
46. Delete its file from object storage. A failure here is logged as a warning but does not fail the operation — an orphaned file wastes storage but breaks nothing, and the response reports honestly whether the file was removed.

The general principle: **fail in the direction that leaves the system consistent**, and report accurately what was and was not done.

12.4 Add Document

Drag-and-drop file upload or URL entry, with an optional title and source name. Uploads are capped at 50 MB, enforced by reading one byte beyond the limit and rejecting if it exists, so an oversized file is refused without being fully buffered.

The response reports the outcome explicitly — processed, duplicate, skipped or failed — with the chunk count and relevance score, so the administrator knows immediately whether the document actually entered the knowledge base and, if not, why.

12.5 Search and Q&A

Search runs raw semantic search over the index: enter a query, see the passages it retrieves with their similarity scores and metadata. This is the primary diagnostic tool. When a user reports that the chatbot could not answer something, the first question is always whether the relevant passages exist and are retrievable — and this page answers it directly.

Q&A runs the full retrieval-and-answer pipeline interactively, with configurable retrieval depth and score threshold, showing the generated answer alongside every retrieved source. It is where prompt behaviour and answer quality can be examined without going through the public interface or consuming quota.

12.6 Chat Quality

Faithfulness monitoring as described in Section 9.4: aggregate statistics, score distribution, and a filterable list of scored answers with their questions.

12.7 Catalogue review

The pending queue and the approved list, each showing an entry's question, answer, sources, grounding score, curation score, served and helpful counts, duplicate count, translation completeness, publication attempts and any publication error — with approve, reject, retract and retry-publish controls.

Surfacing publication errors and attempt counts directly is what makes the asynchronous publication pipeline operable: an entry stuck in `publish_failed` is visible, its cause is readable, and retrying is one click.

12.8 Access Codes

Issuing a code returns its plaintext **once**, at creation, with the interface making clear it cannot be retrieved again — a direct consequence of storing only hashes. The list shows each code's label, organisation, monthly limit, status, current usage and period start. Codes can have their label, organisation, contact message and monthly limit edited, and can be revoked or reactivated.

Revocation is immediate and takes effect on the very next message, since the code's status is checked inside the quota reservation transaction.

12.9 Configuration and health

Configuration shows a read-only snapshot of the running system: LLM provider, model, temperature and token limits; crawl depth, delay, relevance threshold, chunk size and overlap, and any SSL exemptions; which services are configured and at what addresses; which sites are enabled; and the environment. Every secret is masked to `***` or `(not set)`. Read-only is deliberate — configuration changes belong in the deployment's environment, where they are versioned and auditable, not in a web form.

Health performs live checks against PostgreSQL, the vector database, MinIO, the LLM provider, Prefect and Sunbird AI, reporting each as `ok`, `error` or `disabled`. Each check is independent, so one failure does not obscure the others. Translation reports `disabled` rather than `error` when no key is configured, distinguishing an intentional configuration from a fault.

12.10 Crawl triggers

Full and incremental crawls can be started on demand. The system resolves the appropriate Prefect deployment by name — the name incorporates the environment, so a staging console cannot accidentally trigger production's pipeline — and creates a flow run, returning the run identifier.

12.11 Note on the Streamlit-to-React migration

The console was originally a separate Streamlit application. Streamlit is a Python framework for building data interfaces quickly, and it was the right choice for an early internal tool: a functional dashboard in a few hundred lines with no frontend code.

It was replaced by pages in the existing React application for reasons that accumulated:

- **One deployment instead of two.** The Streamlit app was a separate service, container and process. The React pages are routes in an application already deployed.
- **One authentication model.** Streamlit had its own; the React console uses the same JWT-protected API as everything else.
- **Consistent interface.** Administrators and users no longer see two visually and behaviourally different applications.
- **Interaction Streamlit does not do well.** Streamlit re-runs the entire script on every interaction, which makes drag-and-drop upload, live progress, and inline editing awkward.

Because the migration moved logic out of Streamlit into API endpoints, the capabilities became addressable programmatically as well as through a browser — the console is now a client of a documented API rather than a monolith.

The repository retains a vestigial empty `dashboard/` directory, and several documents still describe the Streamlit application. Section 18.3 records this as documentation debt.

13. The Data Model

13.1 Reading a database schema

For readers unfamiliar with relational databases: data is stored in **tables**, which are grids. Each **row** is one record — one document, one message, one access code. Each **column** holds one attribute of that record, with a declared type.

Every table has a **primary key**, a column whose value uniquely identifies a row. This system uses UUIDs — 128-bit random identifiers such as `3f2a...` — rather than sequential numbers, because they can be generated anywhere without coordination and because they do not leak information about how many records exist.

A **foreign key** is a column holding another table's primary key, establishing a relationship: a chunk's `document_id` says which document it belongs to. The database can enforce that such references remain valid, and can be told what to do when the referenced row is deleted — `ON DELETE CASCADE` means dependent rows are removed automatically.

An **index** is an auxiliary structure that makes lookups on a column fast, at the cost of some space and a small penalty on writes. A **unique index** additionally enforces that no two rows share a value.

13.2 Knowledge tables

`crawl_runs` — one row per crawl execution: site, start and completion times, status, counts of URLs discovered and documents processed and failed, a snapshot of the configuration used, and any errors.

`documents` — the central record. Source URL (unique) and site; discovery, processing and update timestamps; publication state (`publicly_visible`, `published_at`, `published_by`, `public_source_url`); title and type; file information (SHA-256 hash, object storage path, size, MIME type); processing status; a content preview; a flexible metadata column; and error tracking with a retry count.

`document_classifications` — one row per document, holding everything the LLM extracted: topics, crops and regions as array columns; relevance score; category; target audience; language; summary; key practices; the actionable, has-data and has-images flags; and which model produced it. Deleting a document cascades to its classification.

`content_chunks` — one row per passage: its document, its index within that document, its text, its estimated token count, metadata inherited from the document's classification, and — critically — `vector_id`, the identifier of its vector in the vector database.

`processing_queue` — pending processing work with priority and retry tracking.

13.3 Conversation tables

`chat_sessions` — one row per conversation: channel (web or WhatsApp), identifier (a generated UUID for web, the phone number for WhatsApp), the owning access grant, and creation and last-active timestamps.

`chat_messages` — every message: its session, role (user or assistant), content, detected language, and four notable additional columns:

- `english_content` — the English original of a translated assistant answer, replayed as conversation history for the token-efficiency reason given in Section 6.2.
- `sources` — the full document-level citation structures, stored as JSON so history can be redisplayed with citations intact without re-running retrieval.
- `suggested_questions` — likewise.
- `bert_score` — the faithfulness measurement, as JSON containing precision, recall, F1, model and timestamp.

13.4 Access tables

`access_codes` — the code's HMAC hash (unique), an optional ciphertext, label, organisation, contact message, monthly limit, status, usage count, period start, and creator.

`access_grants` — the binding: which code, which channel, the identity's hash, status (active or replaced), and redemption and replacement timestamps.

`global_usage_counters` — a single row, id 1, holding the system-wide usage count and period start. A single-row table is an unusual construct, and it is used here deliberately: it gives the global counter a row that can be locked in a transaction exactly like any other, which is what makes the atomic reservation in Section 10.4 possible.

`redemption_attempts` — one row per failed attempt, with a hashed identifier and timestamp, supporting the sliding-window throttle.

A **partial unique index** enforces that the combination of channel and identity hash is unique *among active grants only*. Partial indexes apply a condition, so historical `replaced` grants can coexist while the invariant "one active grant per identity" is guaranteed by the database rather than by application logic.

13.5 Catalogue and job tables

`catalogue_entries` — English question and answer; translations of each as JSON keyed by language; the question's embedding as a float array; status; the source message and source chunk identifiers; grounding and curation scores; served, helpful and duplicate counters; publication error and attempt count; and timestamps.

A partial unique index on `source_message_id` (where it is not null) guarantees that one chat message can produce at most one catalogue entry, making the candidate pipeline safely re-runnable.

`background_jobs` — job type, JSON payload, status, attempt count, an `available_at` timestamp controlling when it may next be claimed, the last error, and timestamps. This is the durable outbox described in Section 14.

`system_metrics` — metric name, value, unit, component, JSON metadata and timestamp.

13.6 The link between PostgreSQL and the vector database

The two stores are joined by a shared identifier. When a chunk is created, a UUID is generated; the chunk row records it as `vector_id`, and the vector is stored under the same identifier with the chunk's text and metadata as its payload.

This dual-write is the system's most delicate consistency point, and it has to work in both directions. Retrieval returns vector identifiers, which the faithfulness scorer resolves back to chunk rows to obtain the exact source text. Deletion must remove both, which is why the delete path removes vectors first (Section 12.3).

13.7 Schema evolution

The system uses **Alembic** to manage schema changes as versioned, ordered migration scripts. Six migrations exist, covering the baseline schema, the addition of BERTScore, chat suggested questions, document publication fields, the access and quota schema, and the catalogue and job schema.

The importance of this is easy to underrate. Once a system holds real data, changing its schema is no longer a matter of editing a definition — every existing deployment must be brought to the new structure without losing anything. Migrations make that reproducible: the same ordered sequence applied to development, staging and production yields the same schema, and each step is reviewable code rather than an ad-hoc command someone ran once.

One detail worth recording because it caused real trouble: the `documents.status` column stores enum values as their **uppercase member names** rather than their lowercase values. The library's raw SQL queries account for this explicitly, with a code comment noting it was verified against the live database. It is the kind of discrepancy that arises when an ORM's enum handling meets hand-written SQL, and documenting it in place is the difference between a five-minute and a five-hour debugging session.

14. The Background Job System

14.1 Why some work cannot happen during a request

Three operations triggered by a chat answer are far too slow to make a user wait:

- **Faithfulness scoring** loads a neural network and runs inference on CPU.
- **Curation assessment** is an additional LLM call.
- **Publication** performs ten translation calls — question and answer, across five languages.

Together these can take a minute or more. The user's answer is ready in seconds and must be delivered then.

14.2 The outbox pattern

The naive solution is to start a background thread. The problem is durability: if the process restarts — a deployment, a crash, a server reboot — every in-flight background task disappears silently. The answer was delivered, the job was lost, and nothing records that it should have happened.

The **outbox pattern** solves this by writing the job to the database **in the same transaction as the data that triggered it**. When a chat turn commits, the messages and the scoring job commit together, atomically. Either both exist or neither does. If the process dies immediately afterwards, the job is sitting in the database waiting, and a worker picks it up whenever one next runs.

Job creation is also **deduplicated**: before adding a scoring job for a message, the system checks whether one already exists for it, so a retried request cannot produce two.

14.3 Safe concurrent claiming

Multiple workers may run. Two workers must not process the same job.

The system uses a PostgreSQL capability designed for exactly this: `SELECT ... FOR UPDATE SKIP LOCKED`. In plain terms:

- **`FOR UPDATE`** — "lock the row I select, so nobody else can take it until I finish."
- **`SKIP LOCKED`** — "and if a row is already locked by someone else, don't wait for it; skip past it and give me the next available one."

Without `SKIP LOCKED`, a second worker asking for the oldest pending job would block until the first released it — serialising all the workers and defeating the point of having several. With it, each worker takes a different job and they proceed in parallel, with the database guaranteeing that no job is ever handed to two workers.

The claim selects the oldest pending job whose `available_at` has passed, marks it running, increments its attempt count, and commits — releasing the lock while retaining ownership through the status change.

14.4 Retries and idempotency

A failed job has its error recorded, and then either:

- **Retries**, if its attempt count is below the maximum of five: status returns to pending and `available_at` is set into the future by an exponentially increasing delay — 2 seconds, 4, 8, 16, capped at 300.
- **Fails permanently**, at five attempts, remaining in the database with its error for inspection.

Exponential backoff matters because most failures are transient and load-related. Retrying immediately, repeatedly, against a service that is rate-limiting or overloaded makes the problem worse. Backing off gives it time to recover.

Retries are only safe because every job type is **idempotent**:

- **Scoring and candidate creation** first checks whether an entry already exists for the message and returns it if so. A retry after a partial success cannot create a duplicate.
- **Publication** commits after each individual translation. A job that fails on the ninth of ten translations resumes with nine already done. It also returns immediately if the entry is already approved.
- **Vector deletion** verifies the entry is actually in the retracted state before acting, and does nothing if the entry no longer exists.

Every outcome — completion, retry, permanent failure — is recorded as telemetry with the job type and attempt count, so worker health is visible in the calibration report.

14.5 The three job types

Type	Triggered by	Work
score_and_catalogue	Every chat answer	Faithfulness scoring, threshold check, curation scoring and question rewriting, embedding, duplicate

		detection, candidate creation
publish_catalogue	Administrator approval, or automatically in auto mode	Ten translations, validation, vector indexing, approval
delete_catalogue_vector	Administrator retraction	Removal of the entry's vector so it stops matching

The worker itself is a small loop: claim a job, run it, repeat; sleep briefly when the queue is empty. It supports a `--once` mode that processes a single job and exits, which is what makes it straightforward to test.

15. Technology Choices and Why

Each subsection states the choice, the alternative, and the reason the alternative lost.

15.1 Python, FastAPI, and Pydantic

Python was effectively determined by the domain. Every library the system depends on for its core work — OpenAI's client, sentence-transformers, BERTScore, Trafilatura, PyMuPDF, yt-dlp, Prefect, Qdrant's and Chroma's clients — is Python-first. Choosing another language would have meant either reimplementing these or operating a polyglot system for no benefit.

FastAPI was chosen over the alternatives (Flask, Django) for three reasons that matter here. It is **asynchronous by default**, which matters for a service whose work is dominated by waiting on external APIs — during a translation call, the server can serve other requests rather than blocking a thread. It **validates automatically** from type annotations, so a malformed request is rejected with a precise error before any handler code runs. And it **documents itself**, generating an interactive API specification from the same annotations.

Pydantic provides that validation, and the system uses it for configuration as well. A single settings class declares every configurable value with its type and default, reads from environment variables and the `.env` file, and fails at startup if something is malformed. A typo in a numeric setting is caught when the process starts, not when the value is first used at three in the morning.

15.2 PostgreSQL over a simpler database

SQLite would have been simpler to operate. It was rejected because the design actively depends on capabilities it does not have:

- **Transactional row-level locking with `FOR UPDATE`** — required for atomic quota reservation (Section 10.4).
- **`SKIP LOCKED`** — required for safe concurrent job claiming (Section 14.3).
- **Array columns with GIN indexes** — topics, crops and regions are naturally lists, and the library filters over them efficiently.
- **JSONB columns** — sources, suggested questions, scores and job payloads are semi-structured; JSONB stores them queryably rather than as opaque text. Faithfulness statistics are computed by aggregating directly over the JSON column.

- **Partial unique indexes** — enforcing "one active grant per identity" while retaining replaced grants.
- **Lateral joins** — pairing each assistant message with its preceding question efficiently.
- **Genuine concurrency** — SQLite's write locking would serialise the API, the worker and the pipeline against each other.

The system uses **SQLAlchemy** as its ORM, and deliberately drops to raw SQL where PostgreSQL can do something the ORM expresses poorly — the faithfulness aggregations, the lateral join, the facet computation with `unnest`. Using an ORM everywhere on principle would have meant fetching data into Python and processing it there, which is slower and harder to read than the SQL it replaces.

15.3 Qdrant and ChromaDB behind one interface

Two vector databases exist for one reason: the friction of local development.

Qdrant is the production choice — purpose-built, fast, with proper filtering, payload storage and operational tooling. But it runs as a separate service, which means a developer must install and run it before they can do anything.

ChromaDB runs *embedded* — a library inside the process, persisting to a local directory. Nothing to install, nothing to start. A developer clones the repository, sets one configuration value, and the system works.

Supporting both without contaminating the codebase uses the **facade pattern**. A `VectorDBClient` presents one interface — ensure collection, upsert, search, delete, get info, check connection — and selects a backend at construction time from configuration. Every other module in the system uses only that interface and has no knowledge of which backend is active.

Two behavioural differences had to be reconciled inside the facade, and they illustrate why the abstraction earns its keep:

Score direction. Qdrant returns a *similarity* where higher is better. Chroma returns a *distance* where lower is better. The Chroma backend converts, so callers always receive a comparable similarity score. Without this, every threshold in the system would mean the opposite thing depending on configuration.

Payload richness. Qdrant stores arbitrary structured payloads. Chroma's metadata must be scalar values. The Chroma backend serialises lists and objects to JSON on write and

parses them back on read, so a caller storing a list of topics gets a list of topics back regardless of backend.

15.4 OpenAI, Groq, and Ollama behind one client

Three LLM providers, one interface, for reasons that changed over time.

OpenAI was the initial choice: reliable, well-documented, and `gpt-4o-mini` is more than adequate for classification and grounded answering. Higher-capability models were not warranted, since the model's job here is to work from supplied evidence rather than to reason from its own knowledge.

Groq was added specifically to reduce cost. It runs open-weight models — Llama 3.3 70B by default — on specialised hardware at substantially lower prices and higher speeds. The integration was cheap because Groq exposes an **OpenAI-compatible API**: the same client library works against a different base URL, so adding Groq required a branch in the client's constructor and nothing else. No changes to generation, streaming, classification, or any service.

Ollama runs models on the system's own hardware at zero marginal cost. It is the fallback when no API key is configured, and it makes fully offline operation possible — which matters both for development without spending money and for any future deployment where sending agricultural questions to a foreign API is unacceptable.

Embeddings remain on OpenAI regardless of the chat provider. Groq offers no embeddings API, and — more importantly — changing embedding models invalidates the entire vector index (Section 2.2). The two decisions are therefore decoupled: the chat model can be switched freely at any time, while the embedding model is effectively fixed for the life of an index.

The client also normalises provider-specific failures into meaningful application errors. A "context too large" rejection arrives differently from OpenAI than from Ollama; the client detects both and raises one specific exception, so the user-facing message can be helpful rather than generic. Failures are retried three times with exponential backoff — except streaming, which is never retried, for the reason given in Section 6.7.

15.5 Embedding model choice

	text-embedding-3-small (OpenAI)	all-MiniLM-L6-v2 (local)
Dimensions	1,536	384

Runs	As an API call	On local hardware
Cost	Per call	Zero marginal
Quality	Higher	Adequate
Data leaves the system	Yes	No
Speed	Network-bound	CPU-bound

The default is OpenAI's, for retrieval quality: a four-fold increase in dimensionality captures finer semantic distinctions, which is precisely what matters when a colloquially phrased question must find formally written technical prose.

The local option matters for three cases: development without incurring cost, deployment in an environment where data sovereignty rules out sending text to an external API, and operation without connectivity.

The constraint that dominates this choice is the one noted in Section 2.2: **all vectors in a collection must come from one model**. Switching means re-embedding every chunk in the corpus. The system enforces consistency by validating vector dimensions before storage and rejecting mismatches loudly rather than corrupting the index. It also detects the specific misconfiguration of naming a local model while the provider is set to OpenAI, warns clearly, and falls back to a valid default.

15.6 Prefect over cron

`cron` runs a command at a time. That is all it does. It offers no retries, no record of what happened, no visibility, and no way to trigger a run manually with different parameters.

Prefect provides all of it without additional code: per-task retries with configurable delays, a persistent record of every run with inputs and outputs, a web interface, on-demand triggering, and parameterised deployments. For a pipeline that spends its time interacting with websites that intermittently fail, this is not a luxury — it is the difference between "the crawl failed" and "the crawl failed at 02:47 on the third of five sites because that server returned 503, and the retry at 02:48 also failed".

The one accommodation Prefect required is worth noting. The lightweight API container ships without Prefect installed, to keep it small. The processing module therefore defines fallback no-op decorators when Prefect cannot be imported, so the identical functions can be called directly — which is what happens when an administrator uploads a document and expects it indexed now rather than at 02:00.

15.7 MinIO

MinIO provides S3-compatible object storage that can be run locally. Nothing in the serving path reads from it; it exists for **provenance and reprocessing**.

Extraction improves. Chunking parameters change. Classification prompts are refined. When any of those happen, the correct response is to reprocess the corpus — and doing so from stored originals takes minutes and costs nothing, whereas re-crawling would take hours, place avoidable load on other people's servers, and might simply fail for documents that have since been moved or withdrawn.

Being S3-compatible means the same code works against MinIO locally and against a cloud object store in production with only a configuration change.

15.8 React, Vite, Tailwind

React for an application that is genuinely interactive — streaming text, live progress, filter state, panels, an administrative console with nine pages. Server-rendered pages would have meant a round trip for every interaction.

Vite as the build tool, for fast development rebuilds and efficient production bundles.

Tailwind CSS for styling, which keeps styles adjacent to the markup they apply to and avoids the drift that accumulates in separate stylesheets. A small set of reusable primitives — button, card, badge, input, select, alert, progress — provides consistency across the public and administrative interfaces.

Supporting libraries are chosen narrowly: `react-router-dom` for routing, `react-markdown` with `remark-gfm` for rendering answers, `recharts` for the console's charts, `react-dropzone` for uploads, `lucide-react` for icons.

15.9 Sunbird AI

Covered in Section 7.2. The short version: for Luganda, Acholi, Runyankole, Ateso and Lugbara, general-purpose translation services are either unavailable or inadequate, because these are low-resource languages that global providers have not prioritised. A Ugandan organisation focused on Ugandan languages is the realistic option, and grounding the system's language capability in local expertise is appropriate to a system built for Uganda.

15.10 Meta Cloud API for WhatsApp

The decision to support WhatsApp at all is the decision that determines the system's reach. A custom mobile application would require users to discover it, install it, create an account, learn it, and keep it updated — and would need storage on a phone that likely has little. SMS cannot carry the length of a useful answer. USSD is worse.

WhatsApp is already installed and already used daily. The Cloud API — Meta's hosted option — requires no messaging infrastructure of the system's own, and its webhook model integrates cleanly with an HTTP service. The dependency on a single commercial platform is a real strategic risk, noted in Section 18.2, and it is accepted because the alternative is not reaching these users at all.

15.11 Docker Compose and Dokku on DigitalOcean

Docker Compose defines the entire local environment — PostgreSQL, MinIO, Qdrant, Prefect server and worker, API, catalogue worker, frontend — in one file, so a developer runs one command and has the system.

Dokku provides production deployment on a single DigitalOcean droplet. It offers the parts of a platform-as-a-service that matter here — git-push deployment, container building, environment management, automatic TLS certificates, zero-downtime restarts — without the operational weight of Kubernetes.

This is right-sized rather than aspirational. The system serves a defined user population, not internet-scale traffic; a single well-specified droplet is sufficient and dramatically simpler to run. The compromises are honest ones: no horizontal scaling, no automatic failover, and a single point of failure. For a dissertation deployment serving a pilot population, these are acceptable; Section 18.2 records them.

15.12 Supporting libraries

Library	Role
Trafilatura	Extracts article text from HTML, discarding navigation and boilerplate
PyMuPDF	Fast PDF text extraction preserving reading order
pdfplumber	PDF table extraction
python-docx	Word document reading, including tables
yt-dlp	Video metadata and transcript retrieval
BeautifulSoup	HTML parsing for link discovery and fallback extraction

httpx	HTTP client with timeouts, redirects and per-request TLS control
SQLAlchemy	ORM and query construction
Alembic	Versioned schema migrations
slowapi	Per-IP rate limiting
tenacity	Retry with exponential backoff
Loguru	Structured logging with rotation and retention
PyJWT	Administrative token issue and verification
bert-score	Answer faithfulness measurement
sentence-transformers	Local embedding generation

16. Running, Deploying, and Operating the System

16.1 Two ways to run it

Full Docker stack. One command starts PostgreSQL, MinIO, Qdrant, the Prefect server and worker, the API, the catalogue worker and the frontend. This is the closest match to production and the right choice for working on the pipeline or the orchestration.

Non-Docker local mode. PostgreSQL and MinIO run natively; the vector database is embedded ChromaDB writing to a local directory; the API, frontend and catalogue worker run as ordinary processes with automatic reload on code changes. This is substantially lighter and is the right choice for working on the API or the interface. A helper script verifies the required local services are reachable before starting anything, so a missing dependency produces a clear message rather than an obscure connection error.

A single command performs setup and start for the non-Docker path.

16.2 Configuration

Every configurable value lives in one settings class, read from the environment. The most consequential are:

Setting	Default	Effect
relevance_threshold	6.0	Bar for indexing and public visibility
chunk_size / chunk_overlap	1000 / 200	Passage granularity
chat_min_top_score	0.30	Confidence required to attempt an answer
catalogue_match_threshold	0.82	Similarity required to serve a stored answer
catalogue_min_faithfulness	0.60	Grounding required for catalogue candidacy
catalogue_min_score	0.70	Curation score required in automatic mode
catalogue_duplicate_threshold	0.90	Similarity at which entries are duplicates
catalogue_suggestion_threshold	0.65	Similarity for related-question suggestions
catalogue_mode	manual	Whether publication requires human approval
api_rate_limit_web	20/hour/IP	Web request ceiling
api_rate_limit_whatsapp	30/hour/phone	WhatsApp message ceiling
global_monthly_message_limit	100,000	System-wide circuit breaker

redemption_max_attempts_per_hour	5	Code-guessing throttle
openai_temperature	0.1	Answer determinism
bertscore_enabled	true	Whether faithfulness scoring runs

Changing a threshold changes the system's behaviour materially, which is the argument for the calibration report of Section 9.6 — these are decisions that should be made from evidence.

16.3 The service inventory

Service	Local URL
Web application	http://localhost:5173
API	http://localhost:8000
Administrative console	http://localhost:5173/admin
Prefect interface	http://localhost:4200
MinIO console	http://localhost:9001
Qdrant	http://localhost:6333
PostgreSQL	localhost:5432 native, localhost:5433 under Docker

PostgreSQL is deliberately exposed on a non-standard port under Docker so it cannot collide with a natively installed instance — a small detail that removes a recurring class of confusing development failure.

16.4 Production deployment

Production runs on a single DigitalOcean droplet managed by Dokku, as two applications: the API and the frontend. The API image is built from a slim Python base and runs uvicorn; the frontend image builds the application with Node and serves the resulting static files with nginx, configured for single-page-application routing and gzip compression.

TLS certificates are issued and renewed automatically. Deployment is by git push. Environment variables are managed per application by Dokku and are never committed to the repository.

16.5 Day-two operations

Health checks. The console's health page exercises every dependency independently. The API also exposes a simple health endpoint for automated monitoring.

Logging. Loguru writes structured logs with automatic rotation at 10 MB and 30-day retention. Log lines carry the identifiers needed to correlate an event across components — document identifiers in the pipeline, session identifiers in chat, phone numbers only where they are not sensitive to log.

Smoke testing. A script exercises the system end to end after deployment, verifying that the major paths work against a live instance.

Where to look when something breaks. The console's health page identifies which dependency is down. The Prefect interface shows pipeline failures with full task-level detail. The documents list shows per-document processing outcomes with explanations. The chat quality page shows whether answer quality has degraded. The calibration report shows whether retrieval is finding less than it used to. These five surfaces cover the large majority of failure modes without reading a log file.

17. Quality Assurance and Reliability

17.1 The test suite

Twenty-nine test modules cover the system, concentrated on the areas where a defect would be most consequential.

Area	Modules	What is verified
Access and quota	test_access_service, test_access_routes, test_access_error_handler, test_access_schema_migration, test_admin_access_routes	Code normalisation and hashing, grant resolution, atomic quota reservation under concurrency, monthly rollover, throttling, the error contract, and schema correctness
Catalogue	test_catalogue_service, test_catalogue_routes, test_catalogue_worker, test_admin_catalogue_routes	Candidate scoring and elimination, duplicate detection, idempotent publication and partial-translation resumption, job claiming and retry, matching and feedback endpoints, review workflow
Chat	test_chat_service, test_chat_routes, test_session_routes	The full pipeline, the refusal path, history windowing, source grouping, authorisation and session ownership, streaming behaviour
WhatsApp	test_whatsapp_webhook, test_whatsapp_service	Signature verification, deduplication, rate limiting, the code-versus-question disambiguation, redemption and rebinding, timeout handling
Pipeline	test_discovery, test_extractors, test_manual_ingestion, test_processing_logs	URL filtering and classification, extraction across formats, ingestion outcomes including duplicates and relevance skips, status explanations
Admin	test_admin_document_delete, test_admin_publication_route, test_admin_ingest_routes, test_admin_crawl_routes, test_admin_document_processing_log	Deletion ordering and failure handling, publication guards, upload limits, crawl triggering
Infrastructure	test_storage, test_llm, test_translation_client, test_utils, test_library_routes, test_operational_metrics	Vector backend parity, provider selection and error normalisation, translation chunking and retries, chunking and cleaning, library visibility rules, metric recording and calibration

The distribution is itself informative: the heaviest coverage is on access control, the catalogue pipeline and the WhatsApp webhook — which are, respectively, where money is at stake, where correctness is hardest, and where inputs are least trustworthy.

17.2 Testing strategy

Tests exercise **real behaviour wherever the behaviour is the system's own**. Database interactions run against a real database, so locking, constraints and transactional semantics are genuinely tested rather than mocked away — which matters enormously for quota reservation, where the whole point is a database-level guarantee.

External services are faked: LLM providers, translation, Meta's messaging API. Their behaviour is not the system's to verify, and depending on them would make the suite slow, expensive and non-deterministic. What *is* tested is how the system responds to their failures — timeouts, malformed responses, rate limits — which is where the system's own logic lives.

17.3 Reliability patterns

The same handful of patterns recur throughout, applied consistently:

Pattern	Where
Retry with exponential backoff	LLM calls, downloads, translation, background jobs
Circuit breaker	Global monthly message ceiling
Graceful degradation	Every optional external dependency (Section 6.10)
Idempotency	Content hashing, catalogue candidacy, publication, job processing
Timeouts	HTTP requests, translation calls, the WhatsApp pipeline
Transactional outbox	Background job creation
Consistent lock ordering	Quota reservation
Fail toward consistency	Document deletion ordering
Bounded caches	WhatsApp deduplication, library facets
Explicit state machines	Document status, catalogue status, job status

17.4 Security posture summary

Concern	Measure
Secrets	Environment variables only, never committed; masked in the configuration view
Access codes	Keyed HMAC-SHA-256 with domain separation; plaintext

	shown once
Phone numbers	Keyed HMAC with a distinct domain; never stored in the clear
Web tokens	256-bit random, SHA-256 hashed at rest
Administrative auth	Signed JWT with expiry; timing-safe credential comparison
Webhook authenticity	HMAC-SHA-256 signature verification, timing-safe
Injection	Parameterised queries throughout; ORM for the general case
Cross-site scripting	Markdown rendered through React; only http(s) links made clickable
Rate limiting	Per IP, per phone, per endpoint
Brute force	Sliding-window redemption throttle
Privacy in logs	Code-shaped messages redacted
Privacy in telemetry	No content, identifiers or exception messages recorded
Information disclosure	Uniform 404s for unpublished documents
Upload safety	50 MB cap enforced without full buffering
Transport	TLS in production; per-site SSL exceptions rather than global disabling

18. Critical Assessment

18.1 What the system does well

It refuses to invent. The retrieval-grounded architecture is not decorative. Retrieval happens before generation; the model receives only retrieved passages; when retrieval finds nothing the model is not called at all. The cheapest and most tempting shortcut — letting the model answer from its own recollection when the corpus falls short — is closed off at the code level.

It measures its own faithfulness. Very few retrieval-augmented systems evaluate their output at all, and fewer still with a signal independent of the model that produced it. Every answer is scored, the scores are visible, and the same scores gate entry to the curated catalogue.

It is honest about its limits, in several senses. It refuses questions it cannot ground. It marks a provisional threshold as provisional in the code. It computes whether it could be trusted to publish without human review and then explicitly declines to act on that computation. It distinguishes "deliberately not indexed" from "failed to index" throughout. This consistency of epistemic posture is unusual.

Its cost discipline is structural rather than bolted on. The browse-first architecture is not a cost optimisation added late; it is the product design. Free browsing, free catalogue matching, metered chat, and a system that gets cheaper to run as it accumulates reviewed answers.

It reaches its users where they are. WhatsApp with a three-word code, in five Ugandan languages, on a phone the user already owns.

It keeps a human in the loop where it matters. Nothing reaches the public library without an administrator publishing it. Nothing enters the catalogue without an administrator approving it. Automated scores are treated as evidence for a person, not as verdicts.

Its failure behaviour is considered. The consistent degradation pattern of Section 6.10, the fail-toward-consistency deletion ordering, the plain-language processing explanations, and the specific rather than generic error messages all reflect attention to what happens when things go wrong — which is where most systems are weakest.

18.2 Known limitations

Retrieval is English-only. The index, the embeddings and the answering all operate in English, with translation at the edges. A Luganda question is translated before search, so retrieval quality depends on translation quality; a mistranslated technical term retrieves the wrong passages, and the user has no way to detect this. The answer is then translated again, compounding the loss. Non-English users therefore receive a measurably weaker service than English users, and the gap is invisible to them.

Translation quality is a hard ceiling. Agricultural and mycological vocabulary — aflatoxin, hermetic storage, moisture content, mycotoxin — is exactly the vocabulary least likely to be well represented in low-resource language training data. A translation that renders a precise threshold imprecisely is worse than an untranslated one, and the system has no mechanism to detect this. There is currently no human review of translated output.

WhatsApp users do not see citations. The medium offers plain text, so the system's central transparency mechanism — showing which documents an answer came from — is unavailable to the audience least equipped to verify claims independently. This is the most significant gap between the system's stated commitments and what a substantial fraction of its users actually receive.

The crawler cannot see JavaScript-rendered content. It parses HTML as served. The Extension Portal is flagged as requiring JavaScript, and APHLIS's data tables and maps are explicitly excluded because they are rendered client-side. Playwright is a project dependency but is not used in the discovery path. Some genuinely valuable material is therefore unreachable.

Scanned PDFs are lost. A large proportion of older and locally produced agricultural documentation is images of pages. Extraction yields nothing and these documents are skipped. `pytesseract` is a declared dependency but is not wired into the extraction path.

Thresholds are judgement, not evidence. The 0.30 retrieval gate, the 0.82 match threshold, the 0.60 faithfulness bar, the 0.70 curation minimum and the 0.90 duplicate threshold were all set by reasoning rather than measurement. The instrumentation to correct this exists and is comprehensive; the data has not yet accumulated. The system knows this about itself and says so, which is the right posture, but the thresholds are provisional and some are almost certainly wrong.

Faithfulness is not correctness. BERTScore confirms that an answer reflects its sources. If a source document is outdated, methodologically weak or simply wrong, a perfectly

faithful answer will propagate the error with full confidence and a citation that lends it authority. Nothing in the system assesses source quality; that burden falls entirely on the human curating the corpus, and the corpus is assembled substantially by automated crawling.

Single-node deployment. One droplet, no horizontal scaling, no automatic failover, one point of failure. Adequate for a pilot, inadequate for a service anyone depends on.

A single administrative credential. One shared username and password, no individual accounts, no roles, no audit trail of who did what. Publication and approval decisions are attributed to a username that everyone shares.

Platform dependency. WhatsApp reach depends entirely on Meta's Cloud API and its commercial terms. Translation depends on a single provider for all five Ugandan languages. Neither has an alternative.

The corpus is small and narrow. Five sources, of which one is disabled by default. Nothing from FAO, CGIAR, regional research networks, or the broader published literature beyond what has been manually ingested. The system's ceiling is its corpus, and the corpus is currently modest.

No feedback on live answers. Users can mark a catalogue answer helpful or unhelpful, but there is no equivalent for live chat answers — which is precisely where user judgement would be most valuable for calibration, since those are the answers no human has reviewed.

18.3 Documentation drift and technical debt

Recorded honestly, since a report that describes only the code and not its surrounding artefacts would be incomplete:

- `README.md` and `docs/ARCHITECTURE.md` describe a Streamlit administrative dashboard that no longer exists, having been migrated into the React application.
- Neither document mentions access control, quotas, the document library, the Q&A catalogue, the ask gate, faithfulness scoring, the background worker, or the calibration report — all of which are substantial parts of the running system.
- `ARCHITECTURE.md` documents retrieval as returning the top 10 passages; the code retrieves 5.
- `ARCHITECTURE.md` states that BERTScore runs as a web-framework background task; it now runs in the separate catalogue worker.

- The `dashboard/` directory survives, containing only an empty directory whose name is an unexpanded shell brace expression — a stray artefact of a `mkdir` that did not expand.
- Two design specifications in `docs/specs/` were partly superseded during implementation, and the implementation plan says so, but the superseded documents are not marked as such at their heads.
- `langchain` and related packages are declared dependencies but appear unused; `pytesseract` and `playwright` are declared and not wired in.

None of this affects the running system. All of it affects anyone trying to understand it from the documentation, which is a real cost for a project intended to be handed over or continued.

18.4 What I would build next

Ordered by expected value relative to effort:

Optical character recognition for scanned PDFs. The dependency is already declared. Adding an OCR fallback when text extraction yields nothing would unlock a meaningful body of material that is currently invisible to the system.

Feedback on live chat answers. A thumbs up/down on every answer would produce, at very low cost, exactly the human-judgement dataset the calibration report needs to move the thresholds from judgement to evidence.

Hybrid retrieval. Semantic search excels at meaning and is weak on exact terms — a specific chemical name, a variety designation, a standard's number. Combining it with keyword search and merging the rankings is a well-established technique that would meaningfully improve retrieval on precisely the queries where the current approach is weakest.

Reranking. Retrieve twenty passages instead of five, then use a cross-encoder — a model that reads the question and a passage together rather than comparing pre-computed vectors — to reorder them and pass the best five to the answering model. This typically produces a substantial retrieval-quality gain for modest additional cost.

Citations on WhatsApp. Even appending source titles and links as a short block after each answer would close the most significant transparency gap in the system.

Voice notes. Speech-to-text on inbound WhatsApp voice notes would remove the literacy barrier for the users who most need the service. Sunbird AI's work includes speech for Ugandan languages, so the provider relationship already exists.

A JavaScript-capable crawler. Playwright is already a dependency. Using it for the sites flagged as requiring JavaScript would reach content currently unreachable.

Corpus expansion. More sources, particularly regional and international research bodies, and a systematic ingestion of the literature already sitting in the repository's `literature/` folder.

Human review of translations. Sampling translated catalogue answers for review by fluent speakers, with corrections stored, would both improve quality and produce data about where machine translation fails in this domain.

Individual administrator accounts with an audit trail. Necessary before the system is operated by more than one person.

19. Conclusion

The dissertation title makes three claims. It is worth closing by testing the system against each.

Retrieval-grounded. This is the claim the architecture most fully supports. Retrieval precedes generation without exception. The answering model receives only passages retrieved from the indexed corpus, together with an instruction to use nothing else and to cite what it uses. When retrieval fails to find anything sufficiently relevant, the model is not invoked at all and the system says plainly that it does not know. Every answer carries its sources back to the user, grouped by document, with similarity scores and links. And — going beyond what the term usually implies — the system independently measures whether each answer actually stayed within its sources, and uses that measurement to gate what enters its permanent curated collection. The grounding is enforced structurally, measured empirically, and visible to the user.

Multilingual. Six languages, with automatic detection, manual override, and coverage extending across the whole product rather than the chat alone. The implementation is honest about its shape: English is the pivot, translation happens at the edges, and non-English users therefore receive a service mediated by two translation steps whose quality the system cannot verify. Catalogue entries mitigate this by being translated once and reviewed as a whole, but the underlying limitation is real and is stated rather than obscured.

An advisory system for post-harvest spoilage management in Uganda. The domain shapes every layer: the sources crawled, the controlled vocabularies of topics and crops, the classification rubric, the answering prompt, the choice of languages, the choice of WhatsApp as a channel, and the decision to issue shared organisational access codes rather than individual accounts. The system is not a general document chatbot with an agricultural corpus attached; it is built around the specific conditions of its users — low-end phones, metered data, limited English, and a genuine need for verifiable rather than merely plausible answers.

Beyond the specific domain, the system demonstrates a set of choices that generalise. That a retrieval-grounded system should refuse rather than improvise, and should be built so that refusing is the cheap path. That an answer's faithfulness to its sources can be measured automatically, and that the measurement is only useful if something acts on it. That the correct posture toward automated quality signals is to treat them as evidence for

a human decision, not as a substitute for one. That a system whose marginal cost is real should meter precisely where the cost occurs and leave everything else open. And that in a domain where being wrong has consequences, an honest "I don't know" is not a failure of the system but one of its most important features.

Appendix A — Glossary

Access code — A three-word string issued by an administrator, redeemable once per identity, carrying a monthly message quota shared by a group.

Access grant — The durable binding created when a code is redeemed, linking it to a specific web token or phone number.

Alembic — The tool used to manage versioned database schema changes.

API — A defined interface through which one program makes requests of another.

BERTScore — A measure of how well a generated answer's meaning is supported by its source texts, computed by matching contextual token representations. Produces precision, recall and F1.

Chunk — A passage of a document, approximately 1,000 tokens, embedded and indexed independently.

ChromaDB — An embedded vector database that runs inside the application process; the development-time backend.

Cosine similarity — A measure of how closely two vectors point in the same direction, used as the similarity score between texts.

Embedding — A list of numbers (1,536 in this system) representing a text's meaning, such that similar texts produce similar lists.

Embedding model — The model that produces embeddings. All vectors in one collection must come from the same model.

Facet — A filterable attribute in the document library — topic, crop or region — presented with counts.

FastAPI — The Python web framework the backend is built on.

Grounding score — This system's term for an answer's BERTScore F1 against its sources.

Hallucination — A fluent, confident, false statement produced by a language model.

HMAC — A keyed hash. Producing or verifying it requires a secret, so hashes cannot be brute-forced by someone holding only the database.

Idempotency — The property that doing something twice has the same effect as doing it once; what makes retrying safe.

JWT — A signed token proving identity, carrying its own expiry, verifiable without a database lookup.

LLM (Large Language Model) — A model trained to predict text, capable of summarising, classifying, following instructions and composing answers.

MinIO — S3-compatible object storage, used here to retain original document files.

Ollama — A tool for running language models on local hardware.

ORM — A library mapping database records to program objects; SQLAlchemy here.

Outbox pattern — Writing a background job in the same database transaction as the data that triggered it, so the job cannot be lost.

Prefect — The workflow orchestration tool that schedules, retries and records pipeline runs.

Qdrant — A standalone vector database; the production backend.

RAG (Retrieval-Augmented Generation) — Retrieving relevant documents and having a language model answer from them, rather than from its own training.

Relevance score — A 0–10 rating of a document's relevance to post-harvest agriculture, produced by the LLM during classification. The threshold is 6.0.

SKIP LOCKED — A PostgreSQL feature allowing multiple workers to claim different rows from a queue without blocking one another.

SSE (Server-Sent Events) — A mechanism for pushing pieces of a response to a browser as they are produced.

Sunbird AI — The Ugandan provider of translation and language identification for the five supported Ugandan languages.

Temperature — A generation parameter controlling randomness. Set to 0.1 here for near-deterministic factual answering.

Token — Roughly a word-fragment; the unit in which language models measure text. Estimated here at four characters per token.

Trafilatura — A library that extracts article text from HTML, discarding navigation and boilerplate.

Vector — See *embedding*.

Vector database — A database that stores vectors and finds the nearest ones to a query vector quickly.

Appendix B — Configuration Reference

Selected settings, with defaults and effects. All are read from environment variables.

Retrieval and answering

Setting	Default	Effect
CHAT_MIN_TOP_SCORE	0.30	Best-match similarity required before an answer is attempted
OPENAI_TEMPERATURE	0.1	Randomness in generation
OPENAI_MAX_TOKENS	8192	Provider request ceiling
EMBEDDING_DIMENSION	1536	Must match the embedding model
EMBEDDING_BATCH_SIZE	32	Texts per embedding request

Pipeline

Setting	Default	Effect
RELEVANCE_THRESHOLD	6.0	Bar for indexing and public visibility
CHUNK_SIZE / CHUNK_OVERLAP	1000 / 200	Passage size and overlap in tokens
MIN_CONTENT_LENGTH	100	Characters below which a document is skipped
MAX_CONTENT_LENGTH	50000	Characters above which text is truncated
CRAWL_MAX_DEPTH	4	Link-hops from a site's start page
CRAWL_DELAY_SECONDS	2.0	Politeness delay between requests
CRAWL_TIMEOUT_SECONDS	30	Per-request timeout
CRAWL_MAX_RETRIES	3	Download retry attempts
CRAWL_UNVERIFY_SSL	—	Comma-separated sites exempt from certificate verification

Catalogue

Setting	Default	Effect
CATALOGUE_MODE	manual	Whether publication requires human approval
CATALOGUE_MATCH_THRESHOLD	0.82	Similarity required to serve a stored answer
CATALOGUE_MIN_FAITHFULNESS	0.60	Grounding required for candidacy
CATALOGUE_MIN_SCORE	0.70	Curation score required in automatic mode
CATALOGUE_DUPLICATE_THRESHOLD	0.90	Similarity at which entries are duplicates
CATALOGUE_SUGGESTION_THRESHOLD	0.65	Similarity for related-question suggestions

CATALOGUE_JOB_MAX_ATTEMPTS	5	Retries before permanent failure
CATALOGUE_AUTO_MIN_REVIEWED_SAMPLES	50	Reviews required before automation is even considered

Access and limits

Setting	Default	Effect
GLOBAL_MONTHLY_MESSAGE_LIMIT	100000	System-wide circuit breaker
REDEMPTION_MAX_ATTEMPTS_PER_HOUR	5	Code-guessing throttle
API_RATE_LIMIT_WEB	20	Requests per hour per IP
API_RATE_LIMIT_WHATSAPP	30	Messages per hour per phone
CATALOGUE_MATCH_RATE_LIMIT	30/hour	Catalogue matching per IP

Providers

Setting	Default	Effect
LLM_PROVIDER	openai	openai, groq or ollama
OPENAI_MODEL	gpt-4o-mini	Answering and classification model
GROQ_MODEL	llama-3.3-70b-versatile	Groq model
OLLAMA_MODEL	llama3.1:8b	Local model
EMBEDDING_PROVIDER	openai	openai or local
EMBEDDING_MODEL	text-embedding-3-small	OpenAI embedding model
VECTOR_DB_PROVIDER	qdrant	qdrant or chroma
TRANSLATION_ENABLED	true	Disabled automatically without an API key
TRANSLATION_MIN_CONFIDENCE	0.80	Detection confidence threshold, applied only when supplied
BERTSCORE_ENABLED	true	Whether faithfulness scoring runs
BERTSCORE_MODEL	distilbert-base-uncased	Scoring model

Appendix C — API Endpoint Reference

Public — no authentication

Endpoint	Purpose
GET /health	Service health
POST /access/redeem	Redeem a code, returns a token once
GET /access/status	Report a token's code status and usage
GET /library/facets	Topic, crop and region values with counts
GET /library/documents	Paginated, filterable published documents
GET /library/documents/{id}	Document detail with bounded excerpts
GET /catalogue	Approved catalogue entries
GET /catalogue/{id}	One entry, in a requested language
POST /catalogue/match	Match a question against the catalogue
POST /catalogue/{id}/helpful	Positive feedback
POST /catalogue/{id}/rejected-match	Negative feedback on a match
GET /chat/models	Available answering models

Access-gated — requires a grant token

Endpoint	Purpose
POST /session/new	Create a chat session
POST /session/from-catalogue	Create a session seeded with a catalogue answer
GET /session/threads	List the caller's conversations
GET /session/{id}/history	Retrieve a conversation
POST /chat	Ask a question
POST /chat/stream	Ask a question, streamed

Webhook — signature-verified

Endpoint	Purpose
GET /webhook/whatsapp	Meta verification handshake
POST /webhook/whatsapp	Inbound messages

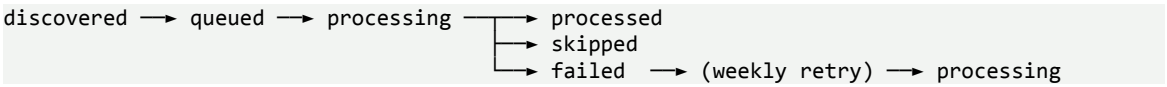
Administrative — requires a JWT

Endpoint	Purpose
POST /admin/login	Exchange credentials for a token
GET /admin/stats	Corpus and activity statistics
GET /admin/documents	Document list with processing outcomes
PATCH /admin/documents/{id}/publication	Publish or unpublish

PATCH /admin/documents/{id}/title	Rename
DELETE /admin/documents/{id}	Delete document, vectors and file
POST /admin/documents/ingest	Ingest a file or URL
POST /admin/search	Raw semantic search
POST /admin/qa	Interactive retrieval and answering
GET /admin/health	Dependency connectivity
GET /admin/config	Configuration snapshot, secrets masked
POST /admin/crawl/full	Trigger a full crawl
POST /admin/crawl/incremental	Trigger an incremental crawl
GET /admin/chat-quality	Scored answers with questions
GET /admin/chat-quality/stats	Faithfulness aggregates
GET /admin/catalogue/pending	Review queue
GET /admin/catalogue/approved	Published and retracted entries
POST /admin/catalogue/{id}/approve	Queue for publication
POST /admin/catalogue/{id}/reject	Reject
POST /admin/catalogue/{id}/retract	Withdraw a published entry
POST /admin/catalogue/{id}/retry-publish	Retry a failed publication
GET /admin/catalogue/calibration	Threshold calibration report
POST /admin/access-codes	Issue a code
GET /admin/access-codes	List codes and usage
PATCH /admin/access-codes/{id}	Edit a code
POST /admin/access-codes/{id}/revoke	Revoke
POST /admin/access-codes/{id}/reactivate	Reactivate

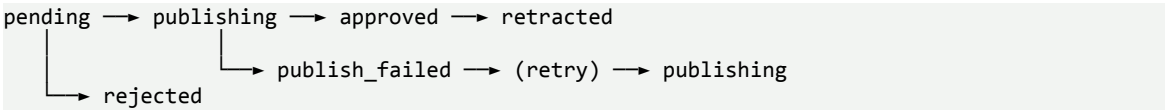
Appendix D — Status Lifecycles

Document status



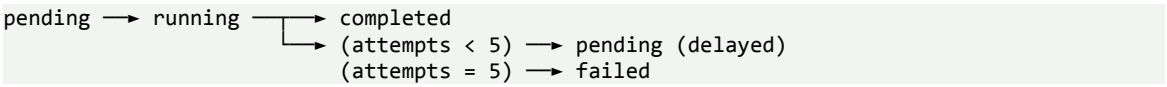
Status	Meaning
discovered	Found and judged relevant by URL; not yet downloaded
queued	Downloaded and stored; awaiting extraction
processing	Extraction, classification and indexing in progress
processed	Successfully indexed and retrievable
skipped	Deliberately not indexed — no usable text, or below the relevance threshold
failed	A pipeline error prevented indexing

Catalogue entry status



Status	Meaning
pending	Candidate awaiting human review
publishing	Approved; translating and indexing
approved	Live and servable
publish_failed	Publication error; retained with its error, retryable
rejected	Declined before publication
retracted	Withdrawn after publication; vector removed

Background job status



Appendix E — Repository Map

Path	Contents
src/api/main.py	Application startup, client initialisation, routing
src/api/routes/	Endpoint handlers: chat, session, whatsapp, library, catalogue, access, admin, admin_access, admin_catalogue, health
src/api/services/chat_service.py	The retrieval-augmented pipeline
src/api/services/access_service.py	Codes, grants, quota reservation, throttling
src/api/services/catalogue_service.py	Candidate scoring, publication, job execution
src/api/services/operational_metrics.py	Telemetry and the calibration report
src/api/services/whatsapp_service.py	Outbound WhatsApp messaging
src/config/settings.py	All configuration
src/config/site_configs.py	Per-site crawl profiles and controlled vocabularies
src/flows/discovery.py	Crawling and URL triage
src/flows/processing.py	Download, extract, classify, chunk, embed, store
src/flows/orchestration.py	Scheduled pipeline composition
src/flows/manual_ingestion.py	File and URL ingestion
src/extractors/	PDF, HTML and video extraction
src/llm/client.py	Unified OpenAI, Groq and Ollama client
src/llm/embeddings.py	Embedding generation
src/llm/prompts.py	Prompt templates
src/llm/bert_scoring.py	Faithfulness scoring
src/storage/models.py	All database models
src/storage/vector_db.py	Vector database facade and both backends
src/storage/postgres.py	Database client
src/storage/minio_client.py	Object storage client
src/translation/client.py	Sunbird AI translation and detection
src/utils/text_processing.py	Chunking and cleaning
frontend/src/ask/	The ask gate
frontend/src/library/	Public document library
frontend/src/catalogue/	Public Q&A catalogue
frontend/src/admin/	Administrative console
frontend/src/access/	Access token context and gate
frontend/src/hooks/	Session and chat state
scripts/catalogue_worker.py	The background worker
scripts/deploy_flows.py	Prefect deployment and schedules
scripts/ingest_document.py	Command-line ingestion
alembic/versions/	Schema migrations
tests/	Test suite
docs/specs/	Design specifications